

SAFE AND SMART MEDICAL DATA EXCHANGE

A PROJECT REPORT

Submitted by

PURUSOTH K R	813821104074
RIYAZ KHAN K I	813821104079
SIVAKARTHIKEYAN D	813821104096
SURYA PRAKASH G	813821104107

*In partial fulfillment for the award of the degree
of*

BACHELOR OF ENGINEERING

in

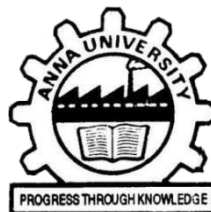
COMPUTER SCIENCE AND ENGINEERING



SARANATHAN COLLEGE OF ENGINEERING

(An Autonomous Institution)

Tiruchirapalli 620 012



ANNA UNIVERSITY :: CHENNAI 600 025

APRIL 2025

SARANATHAN COLLEGE OF ENGINEERING

(An Autonomous Institution)

Tiruchirapalli 620 012

ANNA UNIVERSITY :: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “**SAFE AND SMART MEDICAL DATA EXCHANGE**” is the Bonafide work of “**PURUSOTH K R (813821104074), RIYAZ KHAN K I (813821104079), SIVAKARTHIKEYAN D (813821104096), SURYA PRAKASH G (813821104107)**” who carried out the project work under our supervision.

SIGNATURE

Dr. V. Punitha, M.E., Ph.D.

HEAD OF THE DEPARTMENT

Professor

Department of CSE

Saranathan College of Engineering

Panjapur, Tiruchirappalli – 620012.

SIGNATURE

Ms. G. Roshini, M.TECH

SUPERVISOR

Assistant Professor

Department of CSE

Saranathan College of Engineering

Panjapur, Tiruchirappalli - 620012.

Submitted for the project viva-voce examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We sincerely thank **Shri. S. Ravindran**, Secretary, Saranathan College of Engineering, for giving us a platform to realize our project.

We express our sincere thanks to **Dr. D .Valavan, M.Tech., Ph.D.** Principal of Saranathan College of Engineering for giving an opportunity and immense support for the successful completion of our project.

We owe our special thanks to **Dr. V. Punitha, M.E., Ph.D.** Head of the Department of Computer Science and Engineering for her timely advice, meticulous scrutiny and encouragement which helped us to complete our project. We express our deep gratitude to our project coordinator, **Ms. J Sathiaparkavi, M.E.**, Assistant Professor, Department of Computer Science and Engineering who rendered full support during the period of our project work.

We are extremely thankful to **Ms. G Roshini, M.Tech.**, Assistant Professor, Department of Computer Science and Engineering, for being our internal guide and sharing her knowledge and facilitating valuable guidance for our project.

We would like to thank our parents and friends who have been instrumental in providing ideas and material for the construction of our project and also for their constant encouragement and support. Last but not least we would like to thank the God almighty for all the goodness showered upon us and for the blessing.

ABSTRACT

Security is the most critical issue amid transmission of medical images because it contains sensitive information of patients. Medical image security is an essential method for secure the sensitive data when computerized images and their relevant patient data are transmitted across public networks. Sensitive images carry extensive important information and different features compared to standard images. Medical images have much more sensitive and essential information than any other digital image. Each pixel in the image can be necessary for the diagnosis process, and any deformation can result in a faulty diagnosis. The most robust securing of these images affects an image to the extent that it can be ignored; this is different from insensitive imagery as the border of redundancy is very low. The embedding capacity in medical images is deficient. Existing researchers present different data security techniques as cryptography and data hiding to guarantee data verification. But these approaches take more time and less security in medical image application. So in this project, we implement secure disease diagnosis system for medical images. Experimental results shows that the proposed system implemented Lung CT scan images that are collected from Open medical data sources and with high level security.

TABLE OF CONTENT

CHAPTER NO	TITLE	PAGE NO
	ACKNOWLEDGEMENT	III
	ABSTRACT	IV
	LIST OF TABLES	VIII
	LIST OF FIGURES	IX
	LIST OF ABBREVIATIONS	XI
1	INTRODUCTION	1
1.1	Objective	1
1.2	Scope	1
1.3	Domain	2
1.3.1	Medical Image in cloud	2
1.3.2	Image Processing in Heathcare system	3
1.3.3	Multi Secret Sharing	5
1.3.4	Cryptography	6
1.3.4.1	Symmetric-key Cryptography	7
1.3.4.2	Public-key Cryptography	8
1.3.4.3	Hash Functions	9
2	LITERATURE SURVEY	10
2.1	Biometric-based efficient medical image	10
	Watermarking in E-healthcare application.	

2.2	A novel robust reversible watermarking scheme for protecting authenticity and integrity of medical images	11
2.3	Object detection for medical image	12
2.4	A framework for secure and decentralized Sharing of medical imaging data via block chain consensus	13
2.5	Collaborative learning of semi-supervised Segmentation and classification for medical images	14
2.6	Robust detection of adversarial attacks on medical images	15
2.7	Secure and robust machine learning for healthcare	16
2.8	Lightweight encryption technique to enhance Medical image security on internet medical of applications	17
2.9	A new algorithm for grey and color medical	18
2.10	A lightweight chaos-based medical image encryption scheme using XOR operations	19
3	PROBLEM DEFINITION	20
4	PROPOSED SYSTEM DESIGN	21
4.1	System Architecture	21
5	SYSTEM REQUIREMENT	26
5.1	Hardware Requirements	26

5.2	Software Requirements	26
5.2 .1	Supporting Python Modules	26
5.2.2	Functional Requirements	30
5.2.3	Non-Functional Requirements	31
6	SYSTEM IMPLEMENTATION	33
6.1	Proposed System	33
6.2	Feature Extraction	33
6.3	Performance Evaluation	34
6.4	Implementation	35
7	RESULT AND DISCUSSION	37
7.1	Evaluation and Metrics	37
7.2	Test Cases	38
8	CONCLUSION	39
A	APPENDICES	40
A.1	SAMPLE CODE	40
A.2	SCREENSHOTS	66
	REFERENCES	81

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
7.2	Testcase Table	38

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
4.1	System Architecture Diagram	25
A.1.1	Home page	66
A.1.2	Admin login	67
A.1.3	Health care details	67
A.1.4	Patient details	68
A.1.5	Health care approved details	68
A.1.6	Health care register	69
A.1.7	TTP login	69
A.1.8	Waiting for TTP approval	70
A.1.9	Provide approval	70
A.1.10	Approved healthcare details	71
A.1.11	Healthcare login	71
A.1.12	View approved status	72
A.1.13	New patient register	72
A.1.14	Upload Medical Image and Disease prediction	73
A.1.15	Image split with medical Information hiding	73
A.1.16	Send medical data	74
A.1.17	Patient Login	74
A.1.18	Send key request	75

A.1.19	key request sended	75
A.1.20	Waiting for key permission	76
A.1.21	Health care login	76
A.1.22	Approval for key request	77
A.1.23	Patient Login	77
A.1.24	View shared medical image	78
A.1.25	Secret key verification	78
A.1.26	Images shares decryption	79
A.1.27	View Image shares	79
A.1.28	Shares reconstruction and Medical data extraction	80

LIST OF ABBREVIATIONS

ABBREVIATIONS	DESCRIPTION
CNN	Convolutional Neural Network
LSB	Least Signification Bit
VNA	Vendor Neural Archives
PACS	Picture Archiving and Communication Systems
VPN	Virtual Private Network
CT	Computed Tomography
MG	Mammography Images
MRI	Magnetic Resonance Imaging
PET	Position Emission Tompgraphy
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
SS	Secret Sharing
VSS	Visual Secret Sharing
DES	Data Encryption Standard
AES	Advanced Encryption Standard
VC	Visual Cryptography

VCT	Voluntary Counseling and Testing
HIEA	Hyper Image Encryption Algorithm
ECC	Elliptic Curve Cryptography
GPL	General Public License
HCC	Health Care Center
PL	Patient Login
HL	Healthcare Login
MySQL	My Structured Query Language
WAMP	Windows, Apache, MySQL and PHP
HIPAA	Health Insurance Portability and Accountability Act
GDPR	General Data Protection Regulation
MRA	Magnetic Resonance Angiography
RI	Radioimmunoassay
3DES	Triple Data Encryption Standard
USB	Universal Serial Bus
BM	Brownian Motion
CCS	Chen's Chaotic System
COVID-19	Coronavirus Disease 2019
PIL	Patient Information Leaflet
ROI	Region of Interest
HER	Electronic Health Record

CHAPTER 1

INTRODUCTION

1.1. OBJECTIVE

The purpose of this project is to develop a secure and efficient system for medical image disease classification using Convolutional Neural Networks (CNNs) with multi-secret sharing encryption. The primary goal is to provide healthcare professionals with a reliable tool for automated disease diagnosis while ensuring the confidentiality and integrity of sensitive patient data during transmission and storage. By integrating advanced CNN-based image analysis with robust encryption techniques, the project aims to address the pressing need for secure and accurate medical image analysis in healthcare settings.

1.2. SCOPE

The scope of the project encompasses the following key components:

- Development of CNN models trained on labeled medical image datasets for disease classification.
- Integration of the Least Significant Bit (LSB) algorithm for secure data hiding within medical images to protect patient confidentiality.
- Implementation of multi-secret sharing encryption techniques to split and encrypt medical images for secure transmission and storage.
- Creation of a user-friendly interface for healthcare professionals to input medical images, initiate the classification process, and securely share results.
- Compliance with relevant healthcare regulations and standards for data privacy and security, such as HIPAA and GDPR.
- Documentation and training materials to facilitate seamless adoption and usage of the system by healthcare professionals.

1.3 DOMAIN

1.3.1 MEDICAL IMAGE IN CLOUD

Medical imaging is the technique of creating images of the inner organs of the human body for clinical analysis and medical assistance. With radiography, Medical Resonance Imaging (MRI), ultrasound and many other medical imaging techniques available in the healthcare sector, it is the responsibility of the healthcare industry to keep their patient information safe, and accessible. The standard image storing systems in the healthcare sector are PACS (picture archiving and communication systems) and VNA (Vendor-Neutral Archives). They archive digital imaging from MRIs, X-Rays etc. after a certain period has passed. PACS has an archive option and VNA is used to integrate image files from various PACS into a centralized and cross-platform storage space. With cloud storage, in just the stroke of a button, you are able to deploy complex algorithms that otherwise would have cost huge (for servers and devices) for the industry.

- **Quality of the images** – As these images are very important to both the patient and doctor, it is crucial that they are stored and retrieved as high-definition. Healthcare providers in remote areas have limited access to decent Internet connection. Both quality (of the image) and speed (of the Internet) are essential when a doctor wants to retrieve, say, a radiology image for evaluation.
- **Security** – One cannot complain if some executives in the healthcare sector are reserved about moving to the cloud. Increasing vulnerability to data breaches is a major concern and if the health system is prone to it, most of them won't even consider switching to the cloud. Having said that, an on-premise data centre is vulnerable to damage due to natural disasters, but cloud services will not be affected by any such physical disasters. However, during any cyber-attacks, the on-premise image storage might

prove crucial. The cloud can actually recover easier as they are multi- region. As the cloud offers vast functionality, it is always advisable for the healthcare sector to option for it, making sure that all the data is secure.

- Information sharing – If a healthcare provider is not into using cloud for any of its purposes, then it is, undoubtedly, locked behind a virtual private network (VPN) and firewalls of an on-premise system. This takes huge effort and time for the personnel there to retrieve any records or medical images – a doctor will have to email the radiologist, they have to confirm, there might be some more communication going on until the doctor gets the required medical image. This lengthy and time-consuming process will take place for each patient. Entry of cloud will only ease everything, as one will not have to go through the above process to access a medical image or record.

1.3.2 IMAGE PROCESSING IN HEALTHCARE SYSTEM

In the health care system, there has been a dramatic increase in demand for medical image services, e.g. Radiography, Endoscopy, Computed Tomography (CT), Mammography Images (MG), Ultrasound Images, Magnetic Resonance Imaging (MRI), Magnetic Resonance Angiography (MRA), Nuclear Medicine Imaging, Positron Emission Tomography (PET) and Pathological Tests. Besides, medical images can often be challenging to analyze and time-consuming process due to the shortage of radiologists.

Artificial Intelligence (AI) can address these problems. Machine Learning (ML) is an application of AI that can be able to function without being specifically programmed, that learn from data and make predictions or decisions based on past data. ML uses three learning approaches, namely, supervised learning, unsupervised learning, and semi-supervised learning. The ML techniques include the extraction of features and the selection of suitable features for a specific problem requires a domain expert. Deep Learning (DL) techniques solve the

problem of feature selection. DL is one part of ML, and DL can automatically extract essential features from raw input data. The concept of DL algorithms was introduced from cognitive and information theories. In general, DL has two properties: (1) multiple processing layers that can learn distinct features of data through multiple levels of abstraction, and (2) unsupervised or supervised learning of feature presentations on each layer.

The machine learning algorithm performs a series of preprocessing operations on the initially obtained data and then extracts part of the data from the data generated after the preprocessing program is completed according to the opinions of experts. At the same time, select appropriate data from the data set as the characteristics of the image. After completing a series of operations, the image modeling work is completed by a specific function.

(1) Disease diagnosis – In the field of medical diagnosis and treatment, medical diagnosis is one of the most common medical activities. Medical disease diagnosis also provides a large amount of analytical data for machine learning, which provides conditions for machine learning in the field of medical diagnosis. Machine learning obtains certain data results by sorting out and analyzing a large amount of medical data. Then, a disease diagnosis model is established through machine learning methods, which can provide medical diagnosis assistance to medical diagnosticians.

(2) Medical image processing – As the work of medical institutions becomes more and more difficult, many diseases cannot be diagnosed by traditional clinical solutions, and can only be judged by CT, RI, and other means, resulting in a large number of medical images. However, under current conditions, a large number of medical workers can only analyze these medical images themselves. As medical staff have too much subjective initiative such as medical quality, knowledge level, and personal ability, it is easy to lead to

misunderstanding of medical images, resulting in some wrong diagnosis results. Image processing technology based on machine learning provides a way to effectively avoid the influence of human factors and improve the accuracy of image content interpretation.

1.3.3 MULTI SECRET SHARING

A Secret Sharing (SS) scheme is a cryptosystem that encrypts a secret into multiple pieces called shares so that only qualified sets of shares can be employed to reconstruct the secret. Therefore the SS scheme is one of the most fundamental technologies to realize secure access control. A typical example of secret sharing schemes is a (k, n) -threshold secret sharing scheme. In (k, n) -threshold secret sharing schemes, a secret is encrypted into n shares in such a way that any k or more shares can be employed to reconstruct the secret, while no $k - 1$ or less shares leak any information about the secret. In the ordinary secret sharing schemes, secrets and shares are both numerical data and their encryption and decryption is performed by computers. In contrast, there exist secret sharing schemes whose decryption do not require any numerical computations but can be performed by a human. A Visual Secret Sharing (VSS) scheme is an example of such secret sharing schemes. In VSS schemes, secrets and shares are both visual data such as printed texts, hand written notes, pictures and so on. The schemes encrypt a visual secret into visual shares so that humans can recover the visual secret with their eyes by superposing a qualified set of visual shares printed on transparencies.

Data exchanged over the Internet is in the form of images, audio, video, text, handwritten text, graphic objects, animations etc... The media used in data exchange is unreliable and insecure. Security of the digital media has become an important topic as it can be copied and modified easily. Cryptography is one of the techniques, which can be used for security of exchanged data. It ciphers the plain text to make it as cipher text, which is actually communicated through the

communication media so that intruders even if obtain the cipher text do not be able to decipher the original information hidden within the cipher text. The examples of cryptography are Data Encryption Standard (DES), triple DES (3DES), Advanced Encryption Standard (AES), and Blowfish in which encryption and decryption are done by same key. RSA is another popular algorithm for asymmetric cryptography in which encryption and decryption are done using different keys. Images are a vital form of multimedia contents, which are extensively exchanged over the Internet. So, there should be a secure and simple way to exchange images through any unsecured medium. In order to protect the image contents, Visual Cryptography (VC) is proposed. Using VC, a user can identify confidential data without any computation. In (k, n) -VCT, n shares (shadow images) of the secret image are generated during encryption and are sent through any untrusted medium.

Out of n shares, any k shares are just stacked/superimposed (logical OR operation or AND operation depending upon which color is considered which bit) at the recipient's side to get the original secret image back. Any less than k shares would not regenerate the original secret image. The advantage of VCT over other cryptographical techniques used for other multimedia content like text, audio, video is its decryption process, which does not involve any complex calculations and computations but can only be done by Human Visual System (HVS) i.e. human eye. Furthermore, no key is required for encryption/decryption in VCT as in other cryptographic techniques. The two main aspects of the VCT are contrast and security.

1.3.4 CRYPTOGRAPHY

Cryptography is the science of using mathematics to encrypt and decrypt data. Cryptography enables you to store sensitive information or transmit it across insecure networks (like the Internet) so that it cannot be read by anyone except the intended recipient.

While cryptography is the science of securing data, cryptanalysis is the science of analyzing and breaking secure communication. Classical cryptanalysis involves an interesting combination of analytical reasoning, application of mathematical tools, pattern finding, patience, determination, and luck. Cryptanalysts are also called attackers. Cryptology embraces both cryptography and cryptanalysis.

Cryptography is used in many applications like banking transactions cards, computer passwords, and e-commerce transactions.

Three types of cryptographic techniques used in general.

1. Symmetric-key cryptography
2. Hash functions.
3. Public-key cryptography

1.3.4.1 Symmetric-key cryptography:

Both the sender and receiver share a single key. The sender uses this key to encrypt plaintext and send the cipher text to the receiver. On the other side the receiver applies the same key to decrypt the message and recover the plain text. With symmetric cryptography, the same key is used for both encryption and decryption. A sender and a recipient must already have a shared key that is known to both. Key distribution is a tricky problem and was the impetus for developing asymmetric cryptography. With asymmetric crypto, two different keys are used for encryption and decryption. Every user in an asymmetric cryptosystem has both a public key and a private key. The private key is kept secret at all times, but the public key may be freely distributed.

Data encrypted with a public key may only be decrypted with the corresponding private key. So, sending a message to John requires encrypting that message with John's public key. Only John can decrypt the message, as only John has his private key. Any data encrypted with a private key can only be decrypted with

the corresponding public key. Similarly, Jane could digitally sign a message with her private key, and anyone with Jane's public key could decrypt the signed message and verify that it was in fact Jane who sent it. Symmetric is generally very fast and ideal for encrypting large amounts of data (e.g., an entire disk partition or database). Asymmetric is much slower and can only encrypt pieces of data that are smaller than the key size (typically 2048 bits or smaller). Thus, asymmetric crypto is generally used to encrypt symmetric encryption keys which are then used to encrypt much larger blocks of data. For digital signatures, asymmetric crypto is generally used to encrypt the hashes of messages rather than entire messages. A cryptosystem provides for managing cryptographic keys including generation, exchange, storage, use, revocation, and replacement of the keys.

1.3.4.2 Public-Key Cryptography:

This is the most revolutionary concept in the last 300-400 years. In Public-Key Cryptography two related keys (public and private key) are used. Public key may be freely distributed, while its paired private key, remains a secret. The public key is used for encryption and for decryption private key is used. Public-key cryptography, or asymmetric cryptography, is an encryption scheme that uses two mathematically related, but not identical, keys – a public key and a private key. Unlike symmetric key algorithms that rely on one key to both encrypt and decrypt, each key performs a unique function. The public key is used to encrypt and the private key is used to decrypt. It is computationally infeasible to compute the private key based on the public key. Because of this, public keys can be freely shared, allowing users an easy and convenient method for encrypting content and verifying digital signatures, and private keys can be kept secret, ensuring only the owners of the private keys can decrypt content and create digital signatures. Since public keys need to be shared but are too big to be easily remembered, they are stored on digital certificates for secure transport and sharing. Since private

keys are not shared, they are simply stored in the software or operating system you use, or on hardware (e.g., USB token, hardware security module) containing drivers that allow it to be used with your software or operating system.

1.3.4.3 Hash functions:

No key is used in this algorithm. A fixed-length hash value is computed as per the plain text that makes it impossible for the contents of the plain text to be recovered. Hash functions are also used by many operating systems to encrypt passwords.

CHAPTER 2

LITERATURE SURVEY

2.1 Biometric-based efficient medical image watermarking in E-healthcare application

Authors: Aparna, Puvvadi, and Polurie Venkata Vijay Kishore.

Year: 2019

Propose a biometric-based on an efficient medical image watermarking in E- healthcare application, which produces a system for authentication, confidentiality, and reliability of the system. The proposed system 13ricey13s the fingerprint biometric for authentication, cryptography process for confidentiality, and reversible watermarking for the integrity. Basically, the proposed system consists of two stages such as (i) watermark embedding process and (ii) watermark extraction process. The presented paper intends to reveal the potential of DWM in medical data management issues and propose a novel method to enforce integrity, authenticity, and confidentiality of the medical information, by embedding information bit to the medical image. In this study, at first, we segment the medical image using a region growing (RG) algorithm. Then we encrypt the region of interest (ROI) part using the SHA-256 algorithm. After that, we consider the electric healthcare recode (I), which is having important information about the patient. We have to encrypt the information using the elliptic curve cryptography (ECC) algorithm. For authentication purpose, in this study, we add the fingerprint information. From the fingerprint, we calculate the minutiae point and convert that point into a binary format. Then, we concatenate the Image, fingerprint and I information and compressed using the arithmetic encoding algorithm. Finally, we embed the compressed bit stream into the original image. The same process is repeated for the extraction process. Moreover, the proposed scheme is highly secure having several layers of security mechanisms. Watermarking by datacompression and encryption techniques, embedding of the watermark bits in the embedding region, all these aspects make the proposed scheme an effective novel MIW technique.[1]

2.2 Lightweight encryption technique to enhance medical image for security on internet of medical things applications

Authors: Hasan, Mohammad Kamrul, Shayla Islam, Mohammad Islam.

Year: 2021

Present an efficient, lightweight encryption algorithm for providing secure image encryption in healthcare industry. The proposed lightweight encryption technique employs two permutation techniques to secure medical images. Besides, picture-based information requires more exertion during encryption and decryption. A change procedure dependent on the mix of picture stage and a recently evolved encryption calculation called “Hyper Image Encryption Algorithm (HIEA)”. From the chose picture, we will utilize the twofold worth squares, which will be reworked into a permuted picture using a change procedure, and afterward, the produced picture will be encoded utilizing the “Hyper Image Encryption Algorithm (HIEA)” calculation. All the current strategies utilizing the reasonable client characterized key is created with a similar goal. Likewise, separate between them with a proposed calculation utilized for encryption and decryption. For entropy esteem, connection worth, and execution time of the known cryptographic calculation with proposed cryptography calculations. The proposed lightweight encryption algorithm focused on the efficiency and security of the medical images on IoMT application. The proposed algorithm considered the performance matrix of entropy, as well as correlation. The study has found that the current methods generated key based unsystematic sequence number that creates an enormous computation time. In comparison, it is evident from the result that the proposed algorithm has a small computation. Therefore, to secure the medical image, the proposed algorithm is designed carefully to get optimum plain image.[2]

2.3 A new algorithm for grey and color medial image.

Authors: Kamal, Sara T., Khalid M. Hosny, Taha M. Elgindy, Mohamed M. Darwish, and Mostafa M. Fouda.

Year: 2021

Presents a new encryption algorithm for encrypting both grey and color medical images. A new image splitting technique based on image blocks introduced. Then, the image blocks scrambled using a zigzag pattern, rotation, and random permutation. Then, a chaotic logistic map generates a key to diffuse the scrambled image. Different algorithms for securing medical images are introduced, yet they may be liable to attacks. A strong correlation between neighboring pixels characterizes medical images; thus, removing this correlation requires a permutation (scrambling) technique with a higher security level. Here presents a new algorithm for encrypting medical images that include four parts: image splitting, image scrambling, key generation, and diffusion. First, the plain image is divided into blocks and sub-blocks using a new image splitting technique. Second, the pixels' arrangement is changed in the blocks and sub-blocks using a zigzag pattern, rotation at a 90-degree angle, and random permutation between blocks. Third, a key is generated from the logistic map, where the map's initial using the secret key. A new technique for image splitting is proposed. Random permutation between blocks is applied, and pixels substitution in each block is performed to remove the correlation between pixels. A logistic map is used to diffuse the scrambled image, where the map's initial condition is based on the plain image. Therefore, the proposed algorithm is robust against differential attacks. Analysis of the results proves that our algorithm gains a high performance in encrypting medical images than other methods.[3]

2.4 A novel robust reversible watermarking scheme for protecting authenticity and integrity of medical images

Authors: Liu, Xiyao, Jieting Lou, Hui Fang, Yan Chen, Pingbo Ouyang, Yifan Wang, Beiji Zou, and Lei Wang.

Year: 2019

Liu et al., in their 2019 publication titled “*A Novel Robust Reversible Watermarking Scheme for Protecting Authenticity and Integrity of Medical Images*” in IEEE Access, present a sophisticated technique for embedding watermarks into medical images to ensure both data authenticity and integrity without permanently altering the original content. The core contribution of their work is a reversible watermarking method, which allows the original image to be fully recovered after the watermark has been extracted, thus preserving the clinical quality of medical data — a critical requirement in diagnostic imaging. The proposed scheme uses a combination of histogram modification and prediction-error expansion to embed watermarks in a way that is both imperceptible to human viewers and resilient against common image processing operations. By integrating tamper detection capabilities, the system can alert stakeholders if an image has been altered or corrupted during transmission or storage. This feature is particularly valuable in distributed healthcare systems where image data may traverse multiple platforms or storage units. This work aligns closely with the proposed intelligent lung disease classification system, which embeds diagnosis information into medical images using **LSB** steganography. While Liu et al. emphasize reversibility and integrity verification, the proposed system focuses on confidential information hiding and secure retrieval. Both approaches highlight the importance of embedding information directly into medical images while ensuring the preservation of diagnostic quality and trustworthiness. Liu’s reversible watermarking approach could be further explored to enhance the current system, potentially allowing recovery of both the original image and embedded diagnostic data, offering even higher standards of medical image integrity and security.[4]

2.5 Object detection for medical images

Authors: Li, Zhuoling, Minghui Dong, Shiping Wen, Xiang Hu, Pan Zhou, and Zhigang Zeng.

Year: 2019

Li et al., in their 2019 paper titled “*CLU-CNNs: Object Detection for Medical Images*” published in *Neurocomputing*, introduce an innovative Convolutional Neural Network architecture named CLU-CNNs (Cluster-based Learning Unit CNNs) aimed at improving object detection in medical images. Their approach addresses the challenge of accurately localizing pathological regions—such as tumors, lesions, or infected zones—within complex and often low-contrast medical images. The paper focuses on achieving high detection precision while maintaining computational efficiency, a link for real-time clinical applications. The CLU-CNN framework cluster-based learning strategies to enhance feature representation and ensure more accurate object boundary detection. traditional CNNs, this method divides training data in feature-based clusters, allowing the model to learn localized patterns specifical each cluster. This not only improves detection accuracy but makes the system more adaptable to variations in imaging modalities pathological appearances. The authors demonstrate the effectiveness of their model across several medical imaging datasets, achieving is the superior performance compared to baseline object detection networks. This study supports the development of intelligent medical diagnostic systems, as such as the proposed lung disease classification system using Xception CNN, by showing the importance of specialized CNN architectures for enhanced to image understanding. While the Xception model in the proposed system focuses at on whole-image classification, integrating ideas from CLU-CNNs could refine the framework to know model also the detect and localize disease-specific regions.[5]

2.6 Robust detection of adversarial attacks on medical images

Authors: Li, Xin, and Dongxiao Zhu

Year: 2020

Li and Zhu, in their 2020 paper titled “*Robust Detection of Adversarial Attacks on Medical Images*” presented at the IEEE 17th International Symposium on Biomedical Imaging (ISBI), investigate the growing threat of adversarial attacks on AI systems used in medical imaging. These attacks involve slight, often imperceptible perturbations to input images that can mislead deep learning models, resulting in incorrect or harmful diagnoses. The authors address this vulnerability by proposing a robust detection method that enhances the reliability and trustworthiness of AI-powered diagnostic systems in clinical environments. The proposed framework introduces a combination of statistical analysis and deep learning-based anomaly detection to identify whether an image has been tampered with via adversarial manipulation. Their technique emphasizes minimal computational overhead and high detection accuracy, making it suitable for integration into existing medical imaging pipelines. This is crucial in high-stakes healthcare scenarios, where the misclassification of diseases due to adversarial interference can lead to critical consequences for patients. This study directly supports the development of secure AI-driven healthcare systems, such as the one described in the proposed framework using Xception CNN and steganography. While Li and Zhu focus on ensuring model robustness against attacks, the current system enhances security through encryption, steganography, and edge computing, which also contribute to protecting model integrity and data authenticity. Together, these approaches represent complementary strategies for building resilient and trustworthy AI applications in medical imaging, highlighting the need for both secure input data handling and robust model .[6]

2.7 A lightweight chaos-based medical image encryption scheme using XOR operations

Authors: Masood, Fawad, Maha Driss, and William J. Buchanan

Year: 2021

Implemented a lightweight cryptosystem based on Henon chaotic map, Brownian motion, and Chen's chaotic system to encrypt medical images with elevated security. Here designing an effective multi-stage cryptographic algorithm for medical images encryption using substitution-permutation technique. This multi-stage cryptographic algorithm uses random numbers generated from chaos maps which reduces correlation among the pixels of the digital medical images. Then design a contemporary variant of the chaos-based confusion-diffusion approach that is capable of achieving significant higher entropy and NIST-based randomness results as compared to existing methods. The results demonstrate that the proposed encryption algorithm is able to generate highly secured medical encrypted images. An enhanced image encryption scheme is proposed that combines chaos theory with Brownian motion (BM) and Chen's chaotic system (CCS) to achieve the desired level of security in storage systems of hospitals and medical centers. The proposed system achieves confusion through two- dimensional Henon chaotic map (HCM), whereas 11ricey11s11 is obtained using BM and CCS. Furthermore, the reliability and security of the proposed system are analyzed and compared with existing techniques using the following parameters. The NIST and entropy measures are obtained through randomness test, the consistency and variance through histogram examination, and the pixel similarity using a coefficient of correlation. Other performance analysis parameters include energy, contrast, homogeneity, mean square error, peak to signal noise ratio, number of pixels changing the rate, unified average changing intensity, and computational complexity.[7]

2.8 A framework for secure and decentralized sharing of medical imaging

data via blockchain consensus.

Authors: Patel, Vishal.

Year: 2019

In the paper titled “*A Framework for Secure and Decentralized Sharing of Medical Imaging Data via Blockchain Consensus*” by Vishal Patel (Health Informatics Journal, 2019), the author presents a novel approach to overcoming the challenges of secure medical data sharing using blockchain technology. The study focuses on the need for decentralization and data integrity in medical imaging systems, especially in scenarios where multiple healthcare providers require access to sensitive patient information. Traditional centralized systems are often vulnerable to single points of failure, unauthorized access, and data tampering. To address these concerns, Patel proposes a blockchain-based framework that ensures secure, transparent, and tamper-resistant sharing of medical imaging data. The proposed system employs blockchain consensus protocols to validate and record every access and update to patient imaging data. Smart contracts are used to enforce access control, ensuring that only authorized users can access specific datasets. Furthermore, the immutability of blockchain records provides traceability, enhancing trust between healthcare stakeholders. This approach significantly reduces the risks associated with data breaches and provides a scalable solution for managing distributed medical data repositories. The relevance of this work to the proposed AI-based lung disease detection and steganographic encryption system lies in its emphasis on secure, decentralized data management. While Patel's framework uses blockchain for decentralization and transparency, the proposed system in this study combines edge computing, ECC encryption, and trusted third-party authentication to ensure confidentiality and access control and authenticity.[8]

2.9 Secure and robust machine learning for healthcare: A survey

Authors: Qayyum, Adnan, Junaid Qadir, Muhammad Bilal, and Ala Al- Fuqaha

Year: 2020

Presented an overview of various application areas in healthcare that leverage such techniques from security and privacy point of view and present associated challenges. In addition, we present potential methods to ensure secure and privacy-preserving ML for healthcare applications. Present a comprehensive survey of existing literature on the security and robustness of ML/DL models when used for building healthcare systems with a specific focused on the above- mentioned dimensions. Here note that the aim of this work is to provide an in- depth survey of various security challenges associated with the application of ML/DL in healthcare systems and to provide taxonomy of potential solutions to overcome these issues. Along with discussing security and robustness challenges of using ML/DL models, also briefly elaborate on various general challenges and sources of vulnerabilities that hinder the safe and robust application of ML/DL in healthcare applications. The ML techniques utilizing unlabelled data are known as unsupervised learning methods. Widely used examples of unsupervised learning methods are a clustering of data points using a similarity metric and dimensionality reduction to project high dimensional data to lower dimensional subspaces. The use of machine learning (ML)/deep learning (DL) models for clinical applications has great potential to transform traditional healthcare service delivery. However, to ensure a secure and robust application of these models in clinical settings, different privacy and security challenges should be addressed. Here provided an overview of such challenges by formulating the ML pipeline in healthcare and by identifying different sources of vulnerabilities in it. Finally discussed the potential solutions to provide secure and privacy-preserving ML for security-critical applications like healthcare. [9]

2.10 Collaborative learning of semi-supervised segmentation and classification for medical images.

Authors: Zhou, Yi, Xiaodong He, Lei Huang, Li Liu, Fan Zhu, Shanshan Cui, and Ling Shao.

Year: 2019

In the 2019 paper titled “*Collaborative Learning of Semi-Supervised Segmentation and Classification for Medical Images*”, Zhou et al. present a novel deep learning framework that combines segmentation and classification tasks in a semi-supervised learning environment. Published in the Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), the paper addresses a key limitation in medical imaging: the scarcity of labeled data, which hinders the development of highly accurate AI models. The authors propose a collaborative learning architecture that allows a segmentation model and a classification model to share and learn from each other’s features. This dual-task setup not only improves the performance of both tasks but also makes efficient use of partially labeled datasets. The framework utilizes unlabeled data to guide the learning process, significantly reducing the dependence on large, annotated medical image datasets— an advantage especially important in real-world clinical scenarios where expert labeling is expensive and time-consuming. This research is highly relevant to the proposed lung disease classification system using Xception CNN, which also relies on deep learning for accurate image analysis. While the current system assumes fully labeled training data, Zhou et al.’s approach introduces a way to improve model accuracy in situations with limited annotations. Integrating such a semi-supervised learning strategy into the proposed system could enhance its robustness and scalability across diverse clinical settings. Furthermore, the concept of collaborative learning could be extended to improve both disease detection and segmentation tasks in a unified pipeline, opening avenues for future enhancement of AI-based diagnostic systems. [10]

CHAPTER 3

PROBLEM DEFINITION

The project revolves around the pressing need to ensure the secure transmission and accurate analysis of medical images, critical for effective disease diagnosis and treatment planning in healthcare. Existing methods of sharing medical images often fall short in providing adequate security measures, leaving patient data vulnerable to breaches. Moreover, manual interpretation of medical images for disease diagnosis can be time-consuming and prone to errors. To address these challenges, the project proposes an integrated solution leveraging advanced technologies. Firstly, Convolutional Neural Networks (CNNs) are employed for automated disease classification, enabling efficient and accurate diagnosis from medical images. Additionally, the project incorporates the Least Significant Bit (LSB) algorithm, which facilitates the secure embedding of sensitive patient information within medical images while preserving their visual integrity. This ensures confidentiality during transmission and storage. Furthermore, multi-secret sharing encryption techniques are implemented to enhance security further. By splitting medical images into multiple encrypted shares, each with a different secret key, the system ensures that only authorized parties can access and reconstruct the original image, safeguarding patient privacy and data integrity. Through this comprehensive approach, the project addresses the critical need for secure and accurate medical image analysis, ultimately contributing to improved patient care and outcomes in the healthcare domain.

CHAPTER 4

PROPOSED SYSTEM DESIGN

4.1. SYSTEM ARCHITECTURE

Medical image disease classification using Convolutional Neural Networks (CNN) is a rapidly growing field of research that aims to develop accurate and efficient systems for automated disease diagnosis and treatment. CNNs are deep learning models that have been shown to achieve state-of-the-art results in various computer vision tasks, including medical image analysis. In (fig 4.1) The extracted medical information is hiding within medical image using LSB algorithm. Then the stegno image was applied for multi secret sharing approach. Image sharing with medical data hiding using multi-secret sharing and encryption is a technique used to securely transmit and store medical images by splitting the image into multiple shares, each encrypted with a ECC based different secret key. This technique not only enhances the security and privacy of medical images, but also allows for efficient and reliable sharing of medical data between healthcare professionals.

In the age of digital healthcare, the need for a secure and intelligent system for managing sensitive medical data has become increasingly critical. The proposed architecture presents an integrated solution for the detection and confidential handling of lung disease information using a combination of deep learning, image steganography, encryption techniques, and edge computing. This system ensures that disease classification is both accurate and secure, allowing healthcare professionals to access patient data in a controlled and verified manner. At its core, the system combines artificial intelligence with cybersecurity to address the challenges of data privacy, secure communication, and efficient retrieval. The architecture involves four major stakeholders: healthcare centers, a trusted third party (TTP), edge servers, and a secure processing system with intelligent classification models and encryption

algorithms. The workflow begins at the healthcare center, where patient data and lung images are uploaded into the system. These images are then classified using a machine learning model—specifically, Xception CNN—before the diagnosis is hidden within the image itself using steganography. The hidden information is then encrypted and stored in distributed edge servers for future access. Only authorized entities, verified through a TTP, can retrieve and decrypt this data. The overall goal is to create a decentralized, intelligent, and secure lung disease detection platform that enhances trust, efficiency, and data integrity.

The first technical component of the system is the data selection and disease classification phase. Once the healthcare provider uploads the patient's details and lung images, the images are processed using the **Xception CNN model**, a deep learning architecture known for its ability to extract high-level image features. Xception CNN, which stands for "Extreme Inception," is built upon depthwise separable convolutions, offering both accuracy and computational efficiency. This makes it ideal for real-time disease detection from medical images such as X-rays or CT scans. The lung images are passed through this trained model, which accurately classifies them based on pre-learned patterns indicating various lung conditions—such as pneumonia, tuberculosis, or even COVID-19. This AI-driven classification not only supports early diagnosis but also helps eliminate human errors in medical interpretation. The output of this stage is a textual or coded representation of the diagnosed disease. To prevent the direct exposure of this sensitive diagnosis, the system proceeds to the next phase: steganography-based hiding of the information within the image.

Once the diagnosis is completed, the classified disease information is hidden within the original lung image using the **Least Significant Bit (LSB) algorithm**, a technique from image steganography. LSB works by replacing the least significant bits of pixel values in the image with bits of the disease information, resulting in a visually

unchanged image. This means the image still appears normal to the human eye or standard viewers, but it now carries hidden data that can only be extracted using the correct algorithm and decryption keys. This stegano image not only protects the content from unauthorized viewers but also creates a means to embed secure information without increasing the file size or altering the appearance of the image. This approach is particularly useful in the medical field where image integrity is critical for clinical interpretation. The hidden information can include patient ID, diagnosis, timestamp, and even doctor's remarks. Once the steganographic process is complete, the image is passed into the next stage for splitting and encryption, thereby adding further layers of security.

After steganography, the system employs **multi-secret sharing** to enhance the confidentiality of medical data. In this technique, the stegano image is divided into multiple shares or fragments. Each share contains a partial segment of the hidden data. This ensures that even if an attacker obtains one or more shares, they cannot reconstruct the full image or extract the embedded information without all parts. These shares are then encrypted using **Elliptic Curve Cryptography (ECC)**, a powerful and lightweight encryption method that provides strong security with smaller key sizes. ECC is particularly beneficial in healthcare applications due to its efficiency and lower processing requirements. Each share is encrypted with a unique ECC key before being sent to distributed **edge servers**. These edge servers are located near the healthcare data sources, minimizing latency and enabling quicker access when compared to centralized cloud servers. This distributed architecture not only improves performance but also reduces the risk of single-point failures or centralized attacks.

Once the ECC-encrypted shares are generated, they are securely stored across multiple edge servers. These servers act as local nodes that maintain fragments of the complete image. The decentralized nature of this architecture enhances data resilience and minimizes retrieval delays. The use of edge computing allows real-time

processing and access while maintaining a high standard of privacy and control over sensitive data. However, access to these shares is not open to all. To retrieve the data, a healthcare center must first contact a **Trusted Third Party (TTP)** and submit a request for access. The TTP is responsible for authenticating the identity of the requester and verifying their authorization level. If approved, the TTP generates or releases the corresponding decryption keys required to retrieve the data. This ensures that only verified healthcare providers can access the patient's hidden medical information, protecting it from unauthorized use and privacy breaches. This verification mechanism builds trust and accountability into the data sharing process.

After receiving the decryption keys from the TTP, the healthcare center accesses the encrypted shares from the edge servers. Using the ECC keys, each share is decrypted individually. Once all shares are decrypted, the system merges them back into a single, unified stegano image. At this point, the original image, now reassembled, still contains the hidden disease information embedded using LSB steganography. The final step involves **extracting the hidden data** from this image using the LSB extraction algorithm. This reveals the patient's diagnosis and other relevant metadata. The entire decryption and extraction process ensures that the original patient data remains confidential and is only accessible to authorized personnel at the correct time. This robust multi-layered approach—combining AI, steganography, encryption, and edge computing—demonstrates an innovative and secure methodology for managing medical data. It not only enhances diagnostic accuracy but also ensures that patient privacy and data integrity are never compromised.

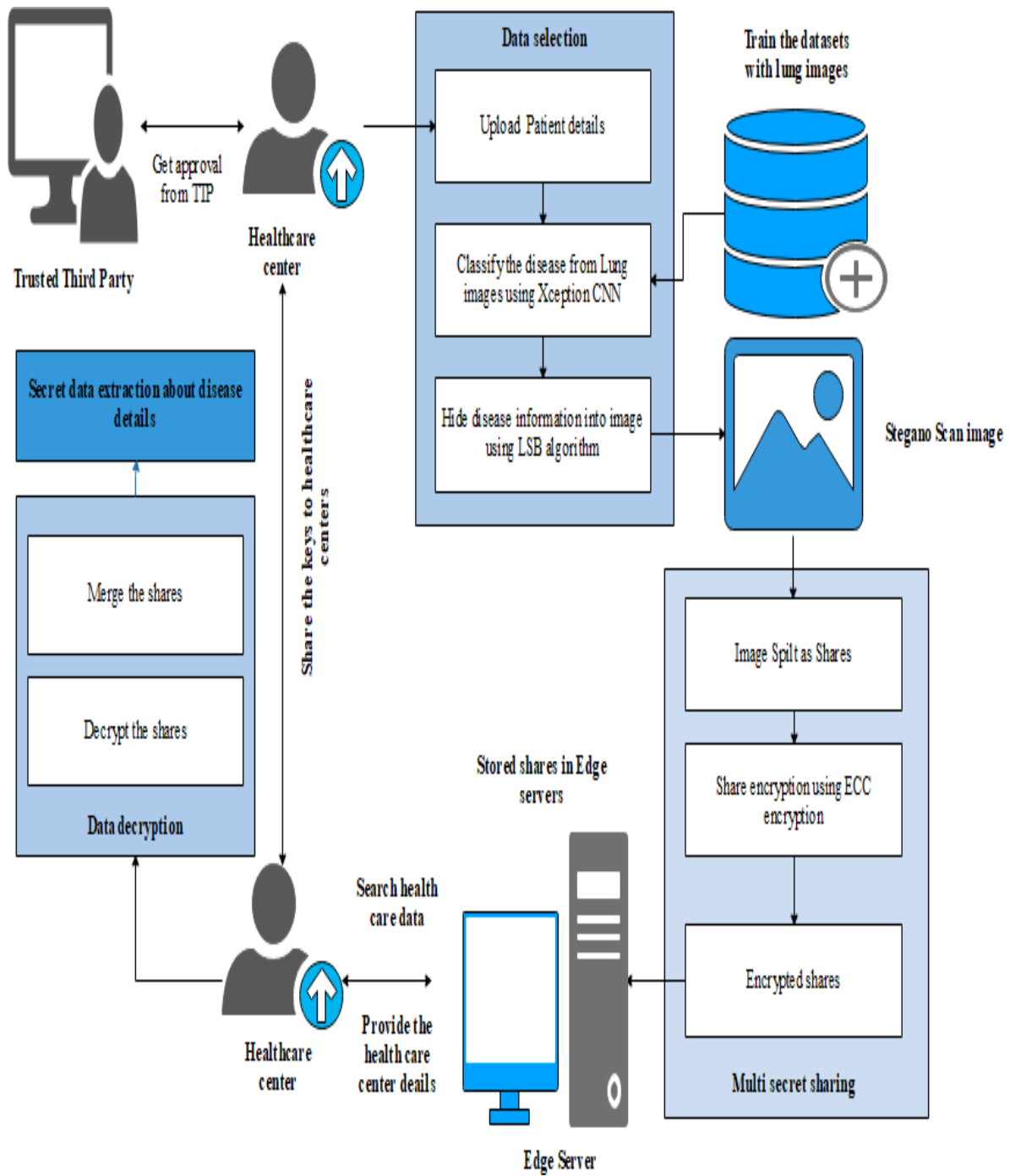


Fig 4.1 System Architecture Diagram

CHAPTER 5

SYSTEM REQUIREMENTS

5.1. HARDWARE REQUIREMENTS

- Processor : Intel processor 2.6.0 GHZ
- RAM : 1GB
- Hard disk : 160 GB
- Compact Disk : 650 Mb
- Keyboard : Standard keyboard
- Monitor : 15 inch color monitor

5.2. SOFTWARE REQUIREMENTS

- Operating System : Windows OS
- Front End : Python
- IDE : Pycharm
- Back End : MySQL
- Application : Web Application

5.2.1. SUPPORTING PYTHON MODULES

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. It was created by Guido van Rossum during 1985-1990. Like Perl, Python source code is also available under the GNU General Public License (GPL). This tutorial gives enough understanding on Python programming language.



Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords

frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages. Python is a MUST for students and working professionals to become a great Software Engineer specially when they are working in Web Development Domain.

Python is currently the most widely used multi-purpose, high-level programming language. Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time. Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc. The biggest strength of Python is huge collection of standard libraries which can be used for the following:

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like OpenCV, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia
- Scientific computing
- Text processing and many more.

➤ **Tensor Flow**

TensorFlow is an end-to-end open-source platform for machine learning. It has a comprehensive, flexible ecosystem of tools, libraries, and community resources that lets researchers push the state-of-the-art in ML, and gives developers the ability to easily build and deploy ML-powered applications.



TensorFlow provides a collection of workflows with intuitive, high-level APIs for both beginners and experts to create machine learning models in numerous languages. Developers have the option to deploy models on a number of platforms such as on servers, in the cloud, on mobile and edge devices, in browsers, and on many other JavaScript platforms. This enables developers to go from model building and training to deployment much more easily.

➤ **Keras**

Keras is a deep learning API written in Python, running on top of the machine learning platform TensorFlow. It was developed with a focus on enabling fast experimentation.

- Allows the same code to run on CPU or on GPU, seamlessly.
- User-friendly API which makes it easy to quickly prototype deep learning models.
- Built-in support for convolutional networks (for computer vision), recurrent networks (for sequence processing), and any combination of both.
- Supports arbitrary network architectures: multi-input or multi-output models, layer sharing, model sharing, etc. This means that Keras is appropriate for building essentially any deep learning model, from a memory network to a neural Turing machine.

➤ **Pandas**

Pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language. Pandas is a Python package that provides fast, flexible, and expressive data structures designed to make working with “relational” or “labeled” data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, real world data analysis in Python. Pandas is mainly used for data analysis and associated manipulation of tabular data in Data frames. Pandas allows importing data from various file formats such as comma-separated values, JSON, Parquet, SQL database tables or queries, and Microsoft Excel. Pandas allows various data manipulation operations such as merging, reshaping, selecting, as well as data cleaning, and data wrangling features. The development of Pandas introduced into Python many comparable features of working with Data frames that were established in the R programming language. The Panda’s library is built upon another library NumPy, which is oriented to efficiently working with arrays instead of the features of working on Data frames.

➤ **NumPy**

NumPy, which stands for Numerical Python, is a library consisting of multidimensional array objects and a collection of routines for processing those arrays. Using NumPy, mathematical and logical operations on arrays can be performed. NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

➤ **Matplotlib**

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. Matplotlib makes easy things easy and hard things possible. Matplotlib is a plotting library for the Python programming language and its numerical mathematics extension NumPy. It provides an object-

oriented API for embedding plots into applications using general-purpose GUI toolkits like Tkinter, wxPython, Qt, or GTK.

➤ **Scikit Learn**

Scikit-learn is a Python module for machine learning built on top of SciPy and is distributed under the 3-Clause BSD license. Scikit-learn are a free software machine learning library for the Python programming language. It features various classification, regression and clustering algorithms including support- vector machines, random forests, gradient boosting, k-means and DBSCAN, and is designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy.

➤ **Pillow**

Pillow is the friendly PIL fork by Alex Clark and Contributors. PIL is the Python Imaging Library by Fredrik Lundh and Contributors. Python pillow library is used to image class within it to show the image. The image modules that belong to the pillow package have a few inbuilt functions such as load images or create new images, etc.

➤ **OpenCV**

OpenCV is an open-source library for the computer vision. It provides the facility to the machine to recognize the faces or objects. In OpenCV, the CV is an abbreviation form of a computer vision, which is defined as a field of study that helps computers to understand the content of the digital images such as photographs and videos.

5.2.2. FUNCTIONAL REQUIREMENTS

➤ **Image Preprocessing**

Accept medical images in standard formats. Preprocess images to enhance quality and standardize format. Extract relevant features or regions of interest for analysis.

➤ **CNN Model Training and Testing**

Train CNN models using labeled datasets of medical images. Implement transfer learning to leverage pre-trained models. Optimize model architecture and hyperparameters for accuracy and efficiency. Evaluate model performance using validation and test datasets.

➤ **LSB Algorithm Integration**

Embed medical information into medical images using the LSB algorithm. Ensure robustness and imperceptibility of hidden data. Allow for reversible extraction of hidden information.

➤ **Multi-Secret Sharing Encryption**

Split medical images into multiple shares using secret sharing schemes. Encrypt each share using elliptic curve cryptography (ECC) or similar techniques. Distribute encrypted shares to authorized parties' securely. Enable reconstruction of the original image using a sufficient number of valid shares.

➤ **System Integration**

Integrate CNN-based image analysis, LSB algorithm, and multi-secret sharing encryption into a cohesive system. Develop user interfaces for inputting medical images, viewing results, and managing encryption keys. Ensure compatibility with existing healthcare information systems and standards.

5.2.3. NON-FUNCTIONAL REQUIREMENTS

The definition for a non-functional requirement is that it essentially specifies how the system should behave and that it is a constraint up on the systems behaviour.

- Performance : Reliability, Accessibility
- Cost : It includes labour cost, miscellaneous cost

Non-functional requirements include the following also:

- The system shall be user friendly and consistent.
- The system shall allow developer access to installed environment.

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1. PROPOSED SYSTEM

Proposed system provides secure medical image sharing with traitor tracing in the encrypted cloud media center. Privacy, integrity, and robustness for the extracted information are important security issues because deep learning enables object detection in images. The Health Care Center (HCC) holds a large volume of medical data and wants to use the cloud for media hosting and sharing. With the continuous development of deep learning technologies, object detection techniques in the medical field have been widely used in many practical medical diagnostic applications. In this project we can implement diagnosis system for Lung images for detecting disease named as COVID or Not. Disease can be predicted using features extraction based CNN algorithm and also hide the disease details in Scan images using LSB coding. Watermarks are required to be securely embedded the medical image features into the image using LSB technique. To prevent data leakage and unauthorized access, the HCC will apply Multi Secret Sharing approach with ECC (Elliptic Curve Cryptography). To share the medical image with an authorized user, the HCC will send the cloud a decryption key to delegate the decryption right. In the encryption key and watermark generation stage, upon receiving the request from a certain user, the HCC produces a watermark (Decryption Key). On receiver side both the decryption key and watermark information could be validated. The authorized user shall be allowed to accessing media object and do not support redistribution process.

6.2. FEATURE EXTRACTION

Feature extraction is an important process in this system, as it allows correct disease detection from lung images by Convolutional Neural Networks (CNN).

The system extracts features, textures, and patterns that are relevant to medical images in order to categorize them as COVID-19 positive or negative. The extracted features assist in distinguishing between healthy lung tissues and infected areas through the analysis of pixel intensities, edges, and deep visual features by using deep learning.

- **Preprocessing** – Input images of lungs are normalized, resized, and augmented to enhance better representation of features.
- **Convolutional Layer Processing** – The model extracts deep features such as edges, shapes, and patterns pertinent in the detection of COVID-19.
- **Feature Selection** – Relevant features are chosen without redundant information, reducing to boost classification accuracy.
- **Classification** – Extracted features are run through a classifier (e.g., Softmax) to classify images as COVID-positive or negative.
- **LSB Encoding** – Disease information is encoded within medical scan images for safe embedding without image distortion.

6.3. PERFORMANCE EVALUATION

The system's performance is evaluated by analyzing its ability to accurately detect COVID-19 cases from lung images while ensuring secure medical data sharing. The evaluation focuses on accuracy, robustness, security, and efficiency.

- **Detection Accuracy** – Measures how accurately the system classifies Lung Disease case
- **Processing Speed** – Evaluates the time taken to process and classify medical images.
- **Security Assessment** – Tests the effectiveness of Multi-Secret Sharing and ECC encryption in protecting medical data.

- **Watermark Robustness** – Examines the ability of the LSB-based watermarking technique to embed and retrieve information without distortion.
- **Scalability** – Checks the system's efficiency in handling a large volume of medical images in the cloud.

6.4. IMPLEMENTATION

Implementation is a process of ensuring that the information system is operational. It involves –

- Constructing a new system from scratch
- Constructing a new system from the existing one.

Implementation allows the users to take over its operation for use and evaluation. It involves training the users to handle the system and plan for a smooth conversion.

➤ **Training**

The personnel in the system must know in detail what their roles will be, how they can use the system, and what the system will or will not do. The success or failure of well designed and technically elegant systems can depend on the way they are operated and used.

➤ **Training Systems Operators**

Systems operators must be trained properly such that they can handle all possible operations, both routine and extraordinary. The operators should be trained in what common malfunctions may occur, how to recognize them, and what steps to take when they come.

Training involves creating troubleshooting lists to identify possible problems and remedies for them, as well as the names and telephone numbers of individuals to contact when unexpected or unusual problems arise.

Training also involves familiarization with run procedures, which involves working through the sequence of activities needed to use a new system.

➤ **User Training**

- End-user training is an important part of the computer-based information system development, which must be provided to employees to enable them to do their own problem solving.
- User training involves how to operate the equipment, troubleshooting the system problem, determining whether a problem that arose is caused by the equipment or software.
- Most user training deals with the operation of the system itself. The training courses must be designed to help the user with fast mobilization for the organization.

➤ **Training Guidelines**

- Establishing measurable objectives
- Using appropriate training methods
- Selecting suitable training sites
- Employing understandable training materials

CHAPTER 7

RESULT AND DISCUSSION

7.1. EVALUATION METRICS

1. **Accuracy (%)** – Measures the correctness of the classification model.
2. **Precision and Recall** – Evaluates the balance between correctly identified positive cases and false negatives.
3. **F1-score** – Harmonic mean of precision and recall to indicate model performance.
4. **Processing Time (ms)** – The time taken for an image to be processed, classified, and secured.
5. **Watermark Robustness (%)** – Measures the ability of LSB watermarking to resist modifications.
6. **Encryption Strength (bits)** – Evaluates the security strength of ECC encryption in protecting medical images.
7. **Storage Efficiency (MB)** – Determines the efficiency of data storage in cloud-based systems.

7.2 TEST CASES

Table 7.2 – Testcase Table

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-01	Upload medical image to the system	Image is uploaded successfully	Pass	
TC-02	Extract features using CNN	Relevant features are extracted from the image	Pass	
TC-03	Classify image as COVID or Non-COVID	Image is correctly classified	Pass	
TC-04	Embed disease details using LSB encoding	Disease details are securely embedded	Pass	
TC-05	Encrypt medical data using ECC	Image is encrypted securely	Pass	
TC-06	Send decryption key to an authorized user	Key is successfully sent to cloud	Pass	
TC-07	Validate watermark on receiver's side	Watermark is retrieved correctly	Pass	
TC-08	Prevent unauthorized access to medical images	Unauthorized users cannot access the image	Pass	
TC-09	Measure system performance on large image datasets	System processes images efficiently	Pass	

CHAPTER 8

CONCLUSION

Secure medical image sharing approach with the combination of cryptography and watermarking techniques was proposed for secure transmission of information through cloud. In this approach disease classification was performed using shared medical image (lung). Then LSB technique is used for watermarking and ECC cryptography is used for share encryption purposes. The proposed technique is not only designed to medical data sharing; however, it is proposed to provide integrity and authentication services for the medical images. Therefore, its target is not to be robust against modification attacks, but its target is to detect any illegal data access. At the receiver side the proposed technique verifies the secret keys shared by HCC regarding illegal access tracing. Proposed techniques provide system authentication service, integrity service and shared information confidentiality service.

A. APPENDICES

A.1 SAMPLE CODE

```
from flask import Flask, render_template, request, session, flash, send_file
from ecies.utils import generate_key
from ecies import encrypt, decrypt
import mysql.connector
import base64, os
```

```
app = Flask(__name__)
app.config['SECRET_KEY'] = 'aaa'
```

```
@app.route('/')
def home():
    return render_template('index.html')
```

```
@app.route('/EdgeServerLogin')
def EdgeServerLogin():
    return render_template('EdgeServerLogin.html')
```

```
@app.route('/TTPLLogin')
def TTPLLogin():
    return render_template('TTPLLogin.html')
```

```
@app.route('/HealthCareLogin')
def HealthCareLogin():
    return render_template('HealthCareLogin.html')
```

```
@app.route('/NewHealthCare')
```

```

def NewHealthCare():
    return render_template('NewHealthCare.html')

@app.route("/ttplogin", methods=['GET', 'POST'])
def ttplogin():
    error = None
    if request.method == 'POST':
        if request.form['uname'] == 'admin' and request.form['password'] ==
'admin':

            conn = mysql.connector.connect(user='root', password="",
host='localhost', database='1SecureObjectdb')
            cur = conn.cursor()
            cur.execute("SELECT * FROM regtb where status='waiting'")
            data = cur.fetchall()

            return render_template('TTPHome.html', data=data)

        else:
            flash("UserName or Password Incorrect!")
            return render_template('TTPLLogin.html')

@app.route("/TTPHome")
def TTPHome():
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM regtb where status='waiting' ") data
= cur.fetchall()

```

```

    return render_template('TTPHome.html', data=data)
@app.route("/RejectInfo")
def TRejectInfo():
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM regtb where status='Rejected' ") data
    = cur.fetchall()
    return render_template('TRejectInfo.html', data=data)

```

```

@app.route("/ApprovedInfo")
def TApprovedInfo():
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM regtb where status='Approved' ")
    data = cur.fetchall()
    return render_template('TApprovedInfo.html', data=data)

```

```

@app.route("/Approved")
def Approved():
    id = request.args.get('id')
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cursor = conn.cursor()
    cursor.execute("Update regtb set Status='Approved' where id='" + id + "' ")
    conn.commit()
    conn.close()

```

```

    conn = mysql.connector.connect(user='root', password='', host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM regtb where status='waiting'")
    data = cur.fetchall()
    flash("Health Care  Approved!")
    return render_template("TTPHome.html", data=data)
@app.route("/Reject")
def Reject():
    id = request.args.get('id')
    conn = mysql.connector.connect(user='root', password='', host='localhost',
database='1SecureObjectdb')
    cursor = conn.cursor()
    cursor.execute("Update regtb set Status='Rejected' where id='" + id + "'")
    conn.commit()
    conn.close()
    conn = mysql.connector.connect(user='root', password='', host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM regtb where status='waiting'")
    data = cur.fetchall()
    flash("Health Care  Rejected!")
    return render_template("TTPHome.html", data=data)
@app.route("/serverlogin", methods=['GET', 'POST'])
def serverlogin():
    error = None
    if request.method == 'POST':
        if request.form['uname'] == 'admin' and request.form['password'] ==

```

```

'admin':
    conn = mysql.connector.connect(user='root', password="",
host='localhost', database='1SecureObjectdb')
    cur = conn.cursor() cur.execute("SELECT
    * FROM regtb ") data = cur.fetchall()

    return render_template('ESeverHome.html', data=data)

else:
    flash("UserName or Password Incorrect!") return
    render_template('ESeverHome.html')

@app.route("/ESeverHome")
def ESeverHome():
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM regtb ")
    data = cur.fetchall()
    return render_template('ESeverHome.html', data=data)

@app.route("/ESeverFileInfo")
def ESeverFileInfo():
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM filetb ")
    data = cur.fetchall()

```

```

return render_template('ESeverFileInfo.html', data=data)

@app.route("/EdgeServerrequest")
def EdgeServerrequest():
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM requesttb ")
    data = cur.fetchall()
    return render_template('EdgeServerrequest.html', data=data)

@app.route("/newhealth", methods=['GET', 'POST'])
def newhealth():
    if request.method == 'POST': name
        = request.form['uname'] mobile
        = request.form['mobile'] email =
        request.form['email']
        website = request.form['website']
        address = request.form['Address']
        username = request.form['username']
        password = request.form['password']

        conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
        cursor = conn.cursor()
        cursor.execute(
            "insert into regtb values('" + name + "','" + mobile + "','" + email + "','"
+ website + "','" + address + "','" + username + "','" + password + "','waiting')")
        conn.commit()

```

```

conn.close()

flash("Record Saved!")


return render_template('NewHealthCare.html')
@app.route("/hclogin", methods=['GET', 'POST'])
def hclogin():
    if request.method == 'POST': username
    = request.form['uname'] password =
    request.form['password']
    session['hname'] = request.form['uname']


    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cursor = conn.cursor()
    cursor.execute("SELECT * from regtb where username='" + username + "'
and password='" + password + "'")
    data = cursor.fetchone()
    if data is None:
        return render_template('index.html')
        return 'Username or Password is wrong'
    else:
        status = data[8]


    if status == 'waiting':
        flash("waiting for TTP Approved")
        return render_template('HealthCareLogin.html', data=data)
    elif status == 'Rejected':

        flash(" Approved Rejected")

```

```

        return render_template('HealthCareLogin.html', data=data)

    else:
        conn = mysql.connector.connect(user='root', password="",
host='localhost', database='1SecureObjectdb')
        cur = conn.cursor()
        cur.execute("SELECT * FROM regtb where username='" + username
+ "' and password='" + password + "'"") data
        = cur.fetchall()

        flash("you are successfully logged in")
        return render_template('HealthCareHome.html', data=data)

@app.route("/upload", methods=['GET', 'POST'])
def upload():
    if request.method == 'POST':
        from stegano import lsb
        from PIL import Image

        name = request.form['uname']
        mobile = request.form['mobile']
        email = request.form['email'] ano
        = request.form['ano'] address =
        request.form['Address']

        import random
        file = request.files['file']
        fnew = random.randint(1111, 9999)
        savename = str(fnew) + ".png"
        file.save("static/upload/" + savename)

```

```

import tensorflow as tf
classifierload = tf.keras.models.load_model('Model/lungmodel.h5')
import numpy as np
from keras.preprocessing import image
test_image = image.load_img("static/upload/" + savename, target_size=(200,
200))
test_image = np.expand_dims(test_image, axis=0)
result = classifierload.predict(test_image)

out = "
pre = "
if result[0][0] == 1:
    out = "Covid"
    pre = "Ivermectin, Hydroxychloroquine"
elif result[0][1] == 1:
    out = "Influenza"
    pre = "Flucelvax Quadrivalent (Seqirus) 5.0-mL MDV"
elif result[0][2] == 1:
    out = "Normal"
    pre = "Nill"
elif result[0][3] == 1:
    out = "Pneumonia"
    pre = "levofloxacin, clarithromycin"
elif result[0][4] == 1:
    out = "Tuberculosis"
    pre = "isoniazid / pyrazinamide / rifampin"

hidedata = "DiseaseName :" + out + " prescription :" + pre

```

```

image = Image.open("./static/upload/" + savename) print(f"Original
size : {image.size}") # 5464x3640

sunset_resized = image.resize((400, 400))
sunset_resized.save("./static/upload/" + savename)

secret = lsb.hide("./static/upload/" + savename, hidedata)

pathname, extension = os.path.splitext("./static/upload/" + savename)
filename = pathname.split("/")
imageName = filename[-1] + ".png"
sname = filename[-1]
secret.save("./static/Encode/" + imageName)

savedir = 'static/Split/'
filename = "./static/Encode/" + imageName
img = Image.open(filename)
width, height = img.size
start_pos = start_x, start_y = (0, 0)
cropped_image_size = w, h = (200, 200)

frame_num = 1
for col_i in range(0, width, w): for
    row_i in range(0, height, h):
        crop = img.crop((col_i, row_i, col_i + w, row_i + h))
        save_to = os.path.join(savedir, sname + "_{:02}.png")
        crop.save(save_to.format(frame_num))
        frame_num += 1

```

```
secp_k = generate_key()
privhex = secp_k.to_hex()
pubhex = secp_k.public_key.format(True).hex()
```

```
filepath1 = "./static/Split/" + sname + "_01.png"
filepath2 = "./static/Split/" + sname + "_02.png"
filepath3 = "./static/Split/" + sname + "_03.png"
filepath4 = "./static/Split/" + sname + "_04.png"
```

```
newfilepath1 = "./static/Encrypt/" + sname + "_01.png"
newfilepath2 = "./static/Encrypt/" + sname + "_02.png"
newfilepath3 = "./static/Encrypt/" + sname + "_03.png"
newfilepath4 = "./static/Encrypt/" + sname + "_04.png"
```

```
data1 = 0
data2 = 0
data3 = 0
data4 = 0
```

```
with open(filepath1, "rb") as File:
    data1 = base64.b64encode(File.read()) # convert binary to string data to
read file
```

```
with open(filepath2, "rb") as File:
    data2 = base64.b64encode(File.read())
```

```
with open(filepath3, "rb") as File:
    data3 = base64.b64encode(File.read())
```

```
with open(filepath4, "rb") as File:
```

```

data4 = base64.b64encode(File.read())

print("Private_key:", privhex, "\nPublic_key:", pubhex, "Type: ",
type(privhex))

if (privhex == 'null'):
    flash('Please Choose Another File,file corrupted!')
    return render_template('Hupload.html')

else:
    encrypted_secp = encrypt(pubhex, data1)
    with open(newfilepath1, "wb") as EFile:
        EFile.write(base64.b64encode(encrypted_secp))

    encrypted_secp = encrypt(pubhex, data2)
    with open(newfilepath2, "wb") as EFile:
        EFile.write(base64.b64encode(encrypted_secp))

    encrypted_secp = encrypt(pubhex, data3)
    with open(newfilepath3, "wb") as EFile:
        EFile.write(base64.b64encode(encrypted_secp))

    encrypted_secp = encrypt(pubhex, data4)
    with open(newfilepath4, "wb") as EFile:
        EFile.write(base64.b64encode(encrypted_secp))

    conn = mysql.connector.connect(user='root', password="",
host='localhost', database='1SecureObjectdb')
    cursor = conn.cursor()

```

```

        cursor.execute(
            "INSERT INTO filetb VALUES ('" + session[
                'hname'] + "','" + name + "','" + mobile + "','" + email + "','" + ano +
                "','" + address + "','" + sname + "','" + savename + "','" + pubhex + "','" + privhex
                + "')"
            conn.commit()
            conn.close()
            flash(hidedata)

        return render_template('HUploadIfo.html', iname=savename, pre=hidedata,
            sname=sname, pvkey=privhex)

@app.route('/Upload')
def Upload():
    return render_template('HUpload.html')

@app.route('/UploadInfo')
def UploadInfo():
    conn = mysql.connector.connect(user='root', password="", host='localhost',
        database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM filetb where HCName='" + session['hname']
        + "'")
    data = cur.fetchall()
    return render_template('HVUploadInfo.html', data=data)

@app.route("/View")
def View():
    id = request.args.get('id')

    conn = mysql.connector.connect(user='root', password="", host='localhost',

```

```

database='1SecureObjectdb')
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM filetb where id='" + id + "'")
    data = cursor.fetchone()

    if data:
        fid = data[8]

    else:
        return 'Incorrect username / password !'

    from stegano import lsb
    clear_message = lsb.reveal("static/Encode/"+fid)
    session['dfid'] = fid

    org = 'static/upload/'+fid

    print(clear_message)
    return render_template('Hview.html', iname=org,pre=clear_message )

@app.route("/hdown", methods=['GET', 'POST'])
def hdown():
    if request.method == 'POST':

        return send_file('static/upload/' + session['dfid'], as_attachment=True)

@app.route('/HSendrequest')
def HSendrequest():
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()

```

```

        cur.execute("SELECT * FROM filetb where HCName !='" +
session['hname'] + "' ")
        data = cur.fetchall()
        return render_template('HSendrequest.html', data=data)

@app.route("/send")
def send():
    id = request.args.get('id')

    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM filetb where id='" + id + "'")
    data = cursor.fetchone()

    if data:
        hname = data[1]
        pname = data[2]
        iname = data[8]
        iid = data[7]
        pkey = data[10]
    else:
        return 'Incorrect username / password !'

    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO requesttb VALUES ('" + id + "', '" + hname + "', '" +

```

```

pname + "','" + iname + "','" + iid + "','" + pkey + "','" + session['hname'] +
"', 'waiting')")
    conn.commit()
    conn.close()

    flash("Key Request Send")
    return render_template('HSendrequest.html')

@app.route('/HStatus')
def HStatus():

    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM requesttb where RHCName ='" +
session['hname'] + "' And Status='waiting' ")
    data = cur.fetchall()

    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM requesttb where RHCName ='" +
session['hname'] + "' And Status='approved' ")
    data1 = cur.fetchall()
    return render_template('HStatus.html', data=data, data1=data1)

@app.route('/HAccept')
def HAccept():

    conn = mysql.connector.connect(user='root', password="", host='localhost',

```

```

database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM requesttb where HCName ='" +
session['hname'] + "' And Status='waiting' ")
    data = cur.fetchall()

    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM requesttb where HCName ='" +
session['hname'] + "' And Status !='waiting' ")
    data1 = cur.fetchall()

    return render_template('HAccept.html', data=data, data1=data1)

@app.route("/rApproved")
def rApproved():
    id = request.args.get('id')

    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM requesttb where id='" + id + "'") data
    = cursor.fetchone()

    if data:
        pkey = data[6]
        rhcname = data[7]
    else:

```

```

        return 'Incorrect username / password !'

    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM regtb where username='" + rhcname +
    """)
    data = cursor.fetchone()

    if data:
        mailid = data[3]

    else:
        return 'Incorrect username / password !'

    msg = "Request Id "+id + " Private key :"+pkey
    sendmsg(mailid ,msg)
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cursor = conn.cursor()
    cursor.execute("Update requesttb set Status='Approved' where id='" + id + "
    ")
    conn.commit()
    conn.close()

    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM requesttb where HCName =" +

```

```

session['hname'] + "' And Status='waiting' ")
    data = cur.fetchall()

    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM requesttb where HCName ='"+
session['hname'] + "' And Status !='waiting' ")
    data1 = cur.fetchall()
    return render_template('HAccept.html', data=data, data1=data1)
@app.route("/rReject")
def rReject():
    id = request.args.get('id')
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cursor = conn.cursor()
    cursor.execute("Update requesttb set Status='Rejected' where id='"+ id + "' ")
    conn.commit()
    conn.close()
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM requesttb where HCName ='"+
session['hname'] + "' And Status='waiting' ")
    data = cur.fetchall()

    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cur = conn.cursor()

```

```

        cur.execute("SELECT * FROM requesttb where HCName ='" +
session['hname'] + "' And Status !='waiting' ")
        data1 = cur.fetchall()

    return render_template('HAccept.html', data=data, data1=data1)

@app.route("/ViewImage")
def ViewImage():
    id = request.args.get('id')

    session["rhcid"] = id

    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM requesttb where id='" + id + "'") data
    = cursor.fetchone()

    if data:
        status = data[8]
    else:
        return 'Incorrect username / password !'
    if status == "Approved":
        return render_template('HDecrypt.html')

    else:
        flash('Your Request Ins Rejected!')
        return render_template('HStatus.html')

@app.route("/imdecrypt", methods=['GET', 'POST'])

```

```

def imdecrypt():
    if request.method == 'POST':
        prikey = request.form['prikey']

        conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1SecureObjectdb')
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM requesttb where id='" + session["rhcid"]
+ "'")
        data = cursor.fetchone()
        if data:
            imid = data[5]
            tpriKey = data[6]

        else:
            return 'Incorrect username / password !' if

        prikey == tpriKey:

        filepath1 = "./static/Encrypt/" + imid + "_01.png"
        filepath2 = "./static/Encrypt/" + imid + "_02.png"
        filepath3 = "./static/Encrypt/" + imid + "_03.png"
        filepath4 = "./static/Encrypt/" + imid + "_04.png"

        newfilepath1 = "./static/Decrypt/" + imid + "_01.png"
        newfilepath2 = "./static/Decrypt/" + imid + "_02.png"
        newfilepath3 = "./static/Decrypt/" + imid + "_03.png"
        newfilepath4 = "./static/Decrypt/" + imid + "_04.png"

```

```

data1 = 0
data2 = 0
data3 = 0
data4 = 0

privhex = tpriKey

with open(filepath1, "rb") as File:
    data1 = base64.b64decode(File.read())

decrypted_secp = decrypt(privhex, data1)

with open(newfilepath1, "wb") as DFile:
    DFile.write(base64.b64decode(decrypted_secp))

with open(filepath2, "rb") as File:
    data2 = base64.b64decode(File.read())

decrypted_secp = decrypt(privhex, data2)

with open(newfilepath2, "wb") as DFile:
    DFile.write(base64.b64decode(decrypted_secp))

with open(filepath3, "rb") as File:
    data3 = base64.b64decode(File.read())
decrypted_secp = decrypt(privhex, data3)
with open(newfilepath3, "wb") as DFile:
    DFile.write(base64.b64decode(decrypted_secp))

```

```

with open(filepath4, "rb") as File:
    data4 = base64.b64decode(File.read())
    decrypted_secp = decrypt(privhex, data4)
    with open(newfilepath4, "wb") as DFile:
        DFile.write(base64.b64decode(decrypted_secp))

    flash('Decrypt Successfully all images')
    return render_template('Hmerge.html', sname=imid)
else:

    flash('Your private key Incorrect!')
    return render_template('HDecrypt.html')

@app.route("/mergeim", methods=['GET', 'POST'])
def mergeim():
    if request.method == 'POST':
        from PIL import Image
        conn = mysql.connector.connect(user='root', password="", host='localhost',
        database='1SecureObjectdb')
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM requesttb where id='" + session["rhcid"]
+ "'")
        data = cursor.fetchone()

        if data:
            imid = data[5]
        else:
            return 'Incorrect username / password !'

```

```

files = [
    "./static/Decrypt/" + imid + "_01.png",
    "./static/Decrypt/" + imid + "_02.png",
    "./static/Decrypt/" + imid + "_03.png",
    "./static/Decrypt/" + imid + "_04.png"]

result = Image.new("RGB", (400, 400))

for index, file in enumerate(files):
    path = os.path.expanduser(file)
    img = Image.open(path)
    img.thumbnail((200, 200), Image.ANTIALIAS)
    x = index // 2 * 200
    y = index % 2 * 200
    w, h = img.size
    print('pos {0},{1} size {2},{3}'.format(x, y, w, h))
    result.paste(img, (x, y, x + w, y + h))

result.save(os.path.expanduser('static/merge/'+ imid +'.png'))
from stegano import lsb
clear_message = lsb.reveal('static/merge/'+ imid +'.png')

mimage = 'static/merge/'+ imid +'.png'
session['mimage'] = mimage

print(clear_message)
return render_template('HDView.html', iname=mimage,
pre=clear_message)

```

```
@app.route("/hvdown", methods=['GET', 'POST'])
def hvdown():
    if request.method == 'POST':

        return send_file(session['mimage'], as_attachment=True)
```

```
def sendmsg(Mailid,message):
    import smtplib
    from email.mime.multipart import MIMEMultipart
    from email.mime.text import MIMEText
    from email.mime.base import MIMEBase
    from email import encoders
```

```
fromaddr = "sampletest685@gmail.com" toaddr
= Mailid
```

```
# instance of MIMEMultipart
msg = MIMEMultipart()
```

```
# storing the senders email address
msg['From'] = fromaddr
```

```
# storing the receivers email address
msg['To'] = toaddr
```

```
# storing the subject
msg['Subject'] = "Alert"
```

```
# string to store the body of the mail
```

```
body = message

# attach the body with the msg instance
msg.attach(MIMEText(body, 'plain'))

# creates SMTP session
s = smtplib.SMTP('smtp.gmail.com', 587)

# start TLS for security
s.starttls()

# Authentication
s.login(fromaddr, "hneucvnontsuwgpj")

# Converts the Multipart msg into a string
text = msg.as_string()

# sending the mail
s.sendmail(fromaddr, toaddr, text)

# terminating the session
s.quit()

if __name__ == '__main__':
    app.run(debug=True, use_reloader=True)
```

A.2 SCREENSHOTS

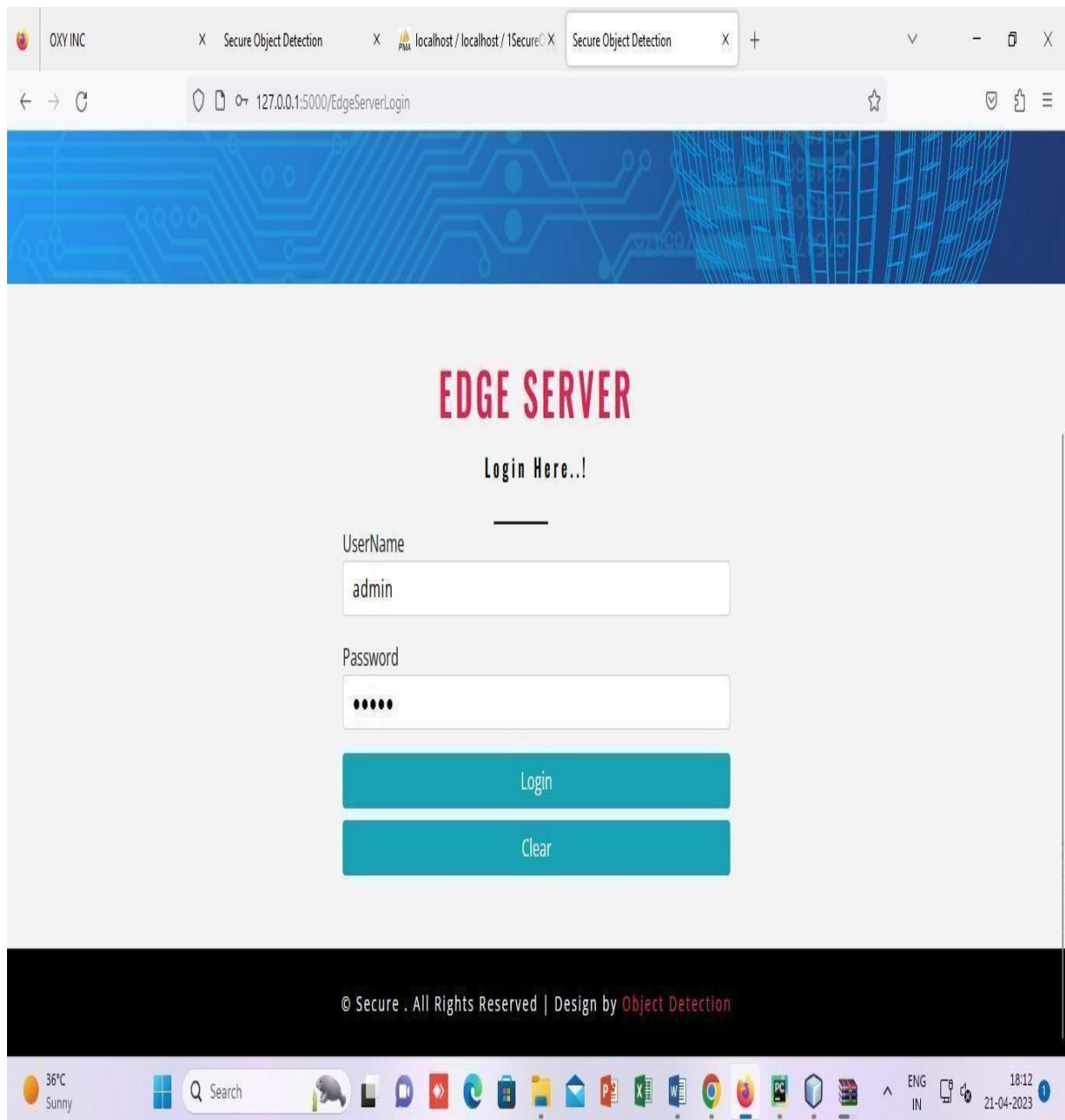


Fig 1.1 Homepage

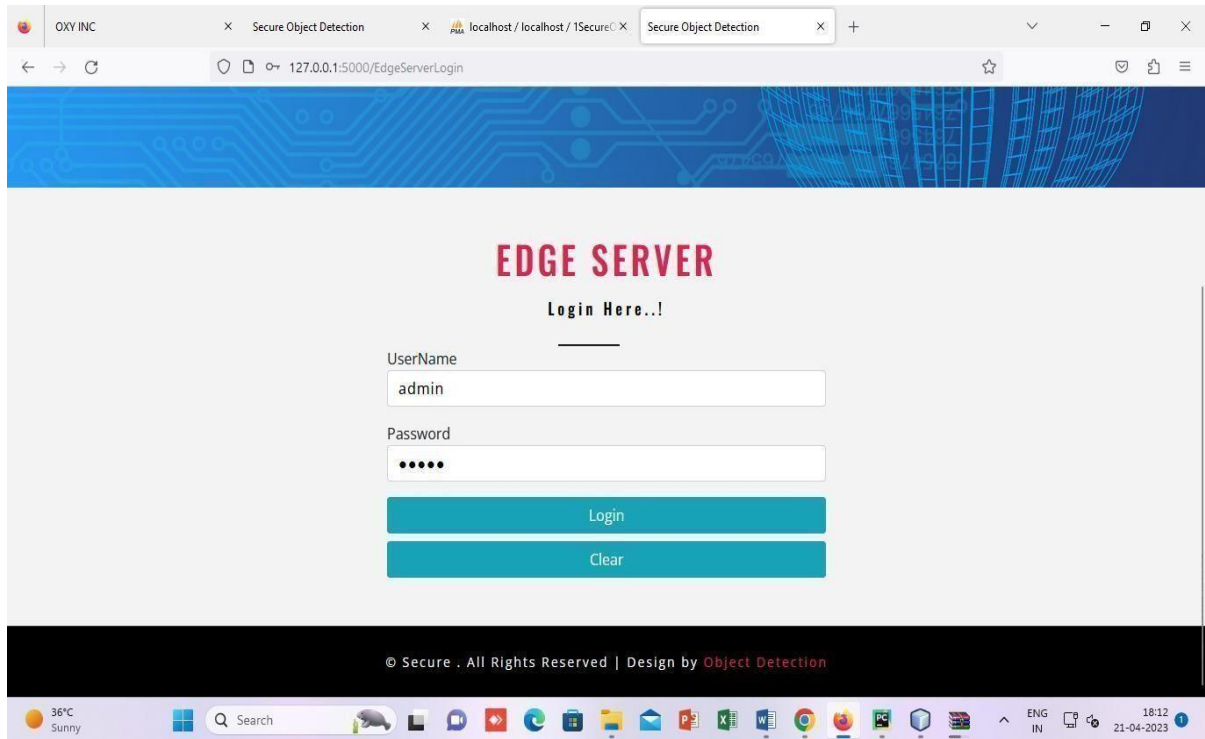


Fig 1.2 Admin Login

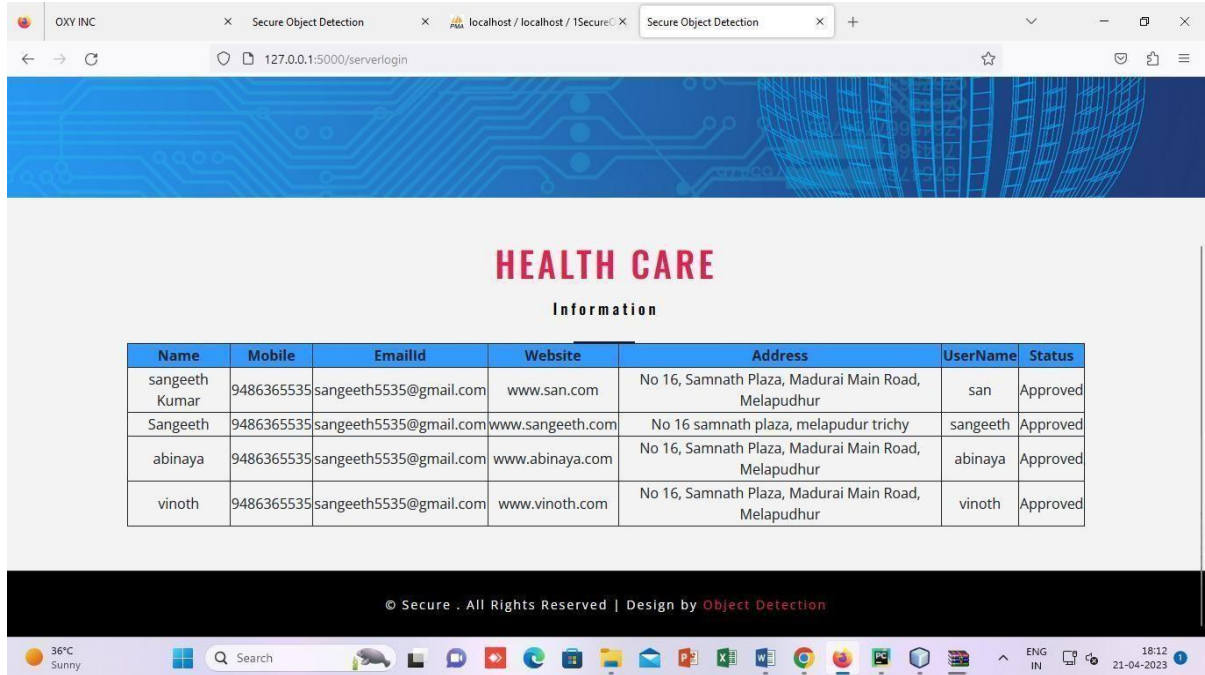


Fig 1.3 Health Care Details

PatientName	Mobile	EmailId	AadharNo	FileName	Public Key
admin	9486365535	sangeeth5535@gmail.com	948636553535	1445.png	025a98f19b2c09d38a7c242bd4e33fc4e735d8e4e5b8928cabcd8f3912caf8471
admin	9486365535	sangeeth5535@gmail.com	948636553535	5446.png	027bdc6380c5644a4cb308487f38e133e978d8258973186ffe217036c9f283ecc7
admin	9486365535	sangeeth5535@gmail.com	948636553535	4238.png	0225110b1f4edfd75debb0dd9ca97396b9b16f8fc6a792abef1d8e20f480e014b6
admin	9600357839	geetha@gmail.com	948636553535	4994.png	03219abb15d335fadd7efbf18994b6ed9c18823bb9820d1a1d13540cb64a63c49
sangeeth	9486365535	sangeeth5535@gmail.com	948636553535	4449.png	03b9b01aefb6c0d7a7c7d49d70d974afccf438ac2767920827b73492704fe2963
sangeeth	9486365535	sangeeth5535@gmail.com	346457563464	4192.png	03309885433f295b718ae535c17de4fdbc540c1c0545bfee84a57edcf01b7fb94
sangeeth	9486365535	sangeeth5535@gmail.com	235234645788	4204.png	03b71939795ac5865cc893886d90562e395b2c34f8399552036f6c1836fa4e0f60

© Secure . All Rights Reserved | Design by Object Detection

Fig 1.4 Patient Details

RequestId	HealthCareName	PatientName	ImageName	Status
1	sangeeth	admin	4238.png	Approved
2	abinaya	sangeeth	4449.png	Approved

© Secure . All Rights Reserved | Design by Object Detection

Fig 1.5 Health Care Approval Details

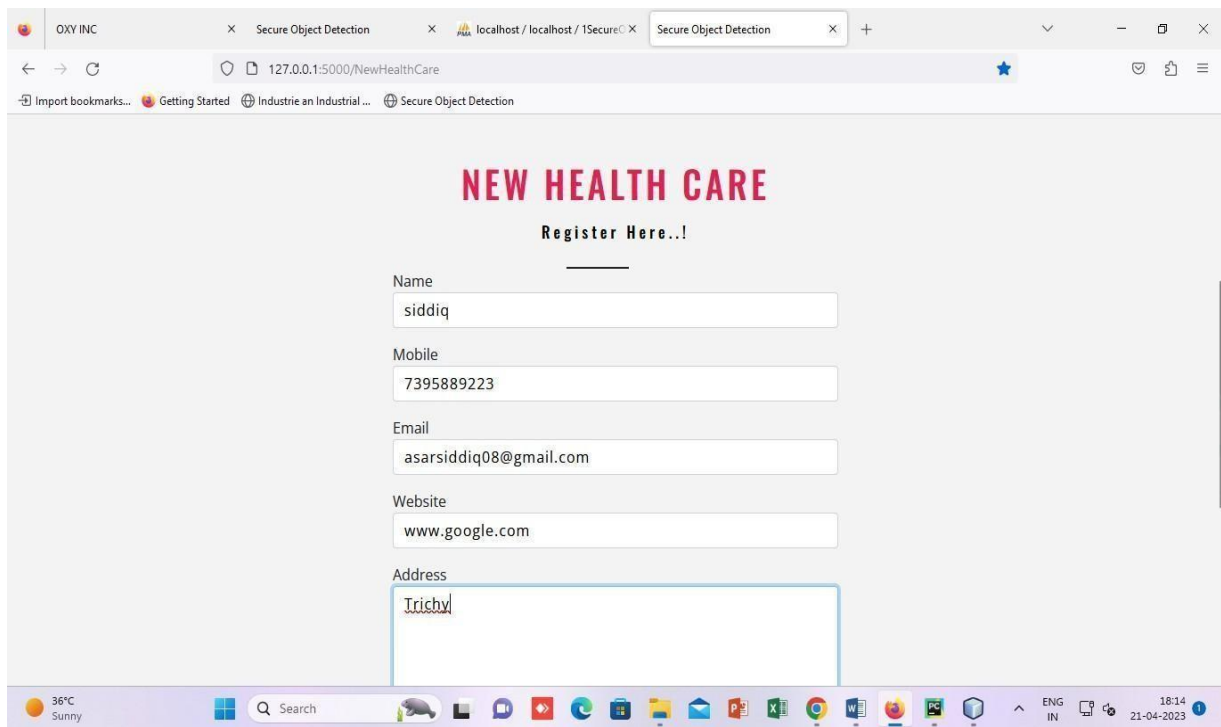


Fig 1.6 Health Care Register

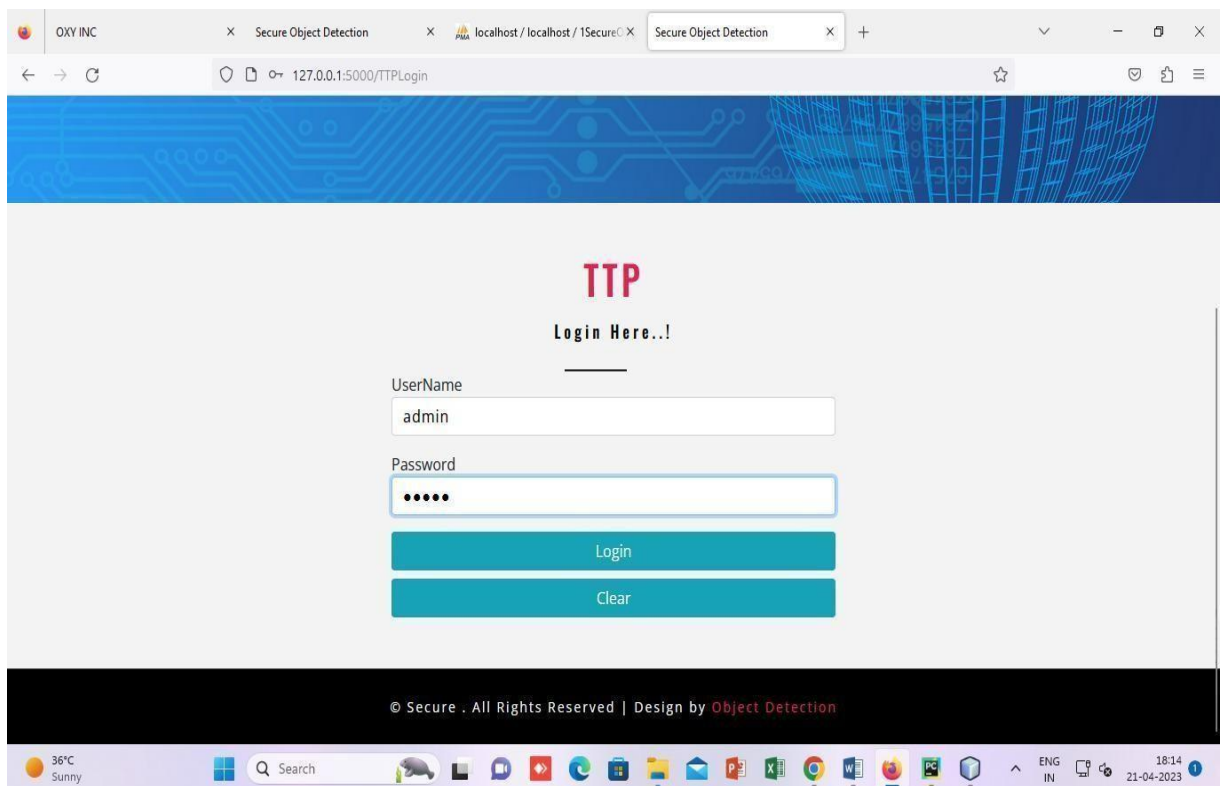


Fig 1.7 TTP Login

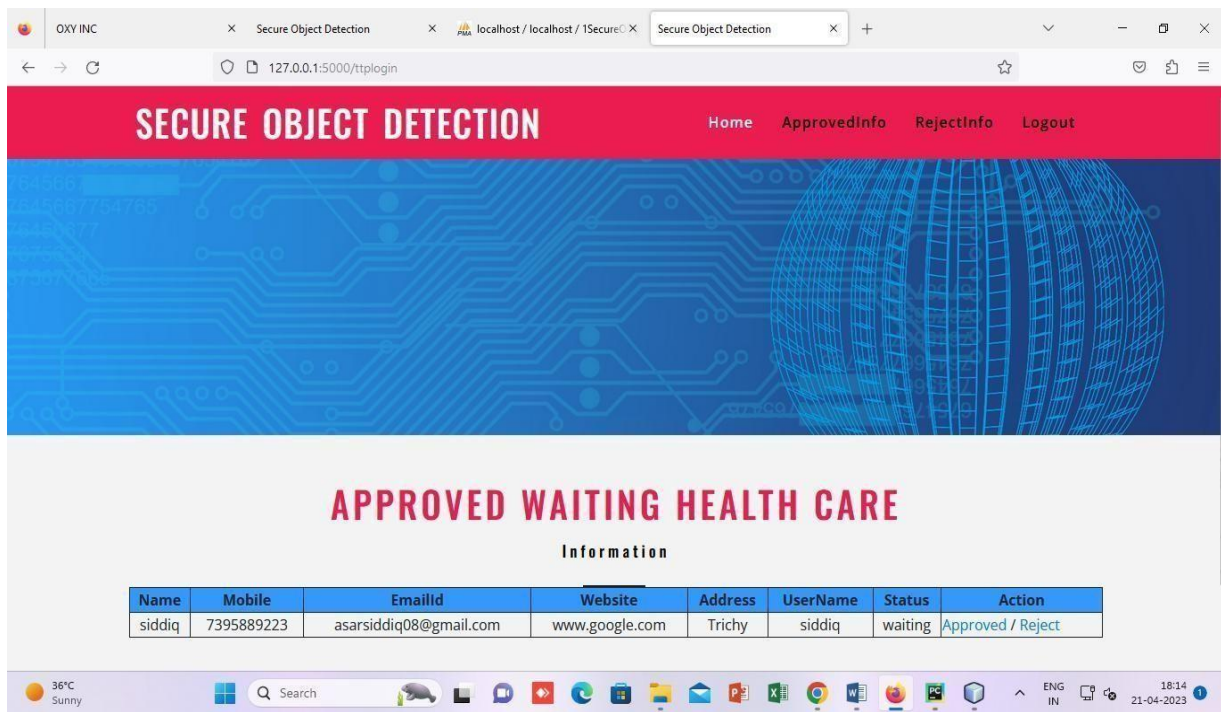


Fig 1.8 Waiting for TTP Approval

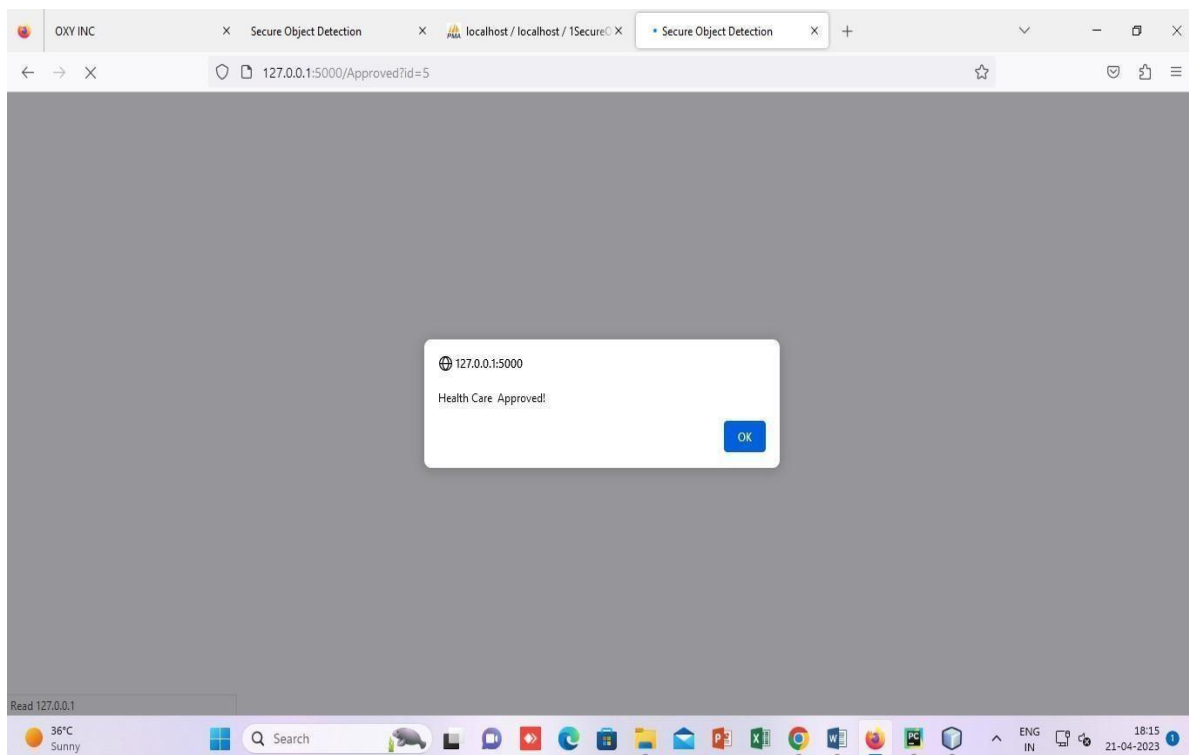


Fig 1.9 Provide Approval

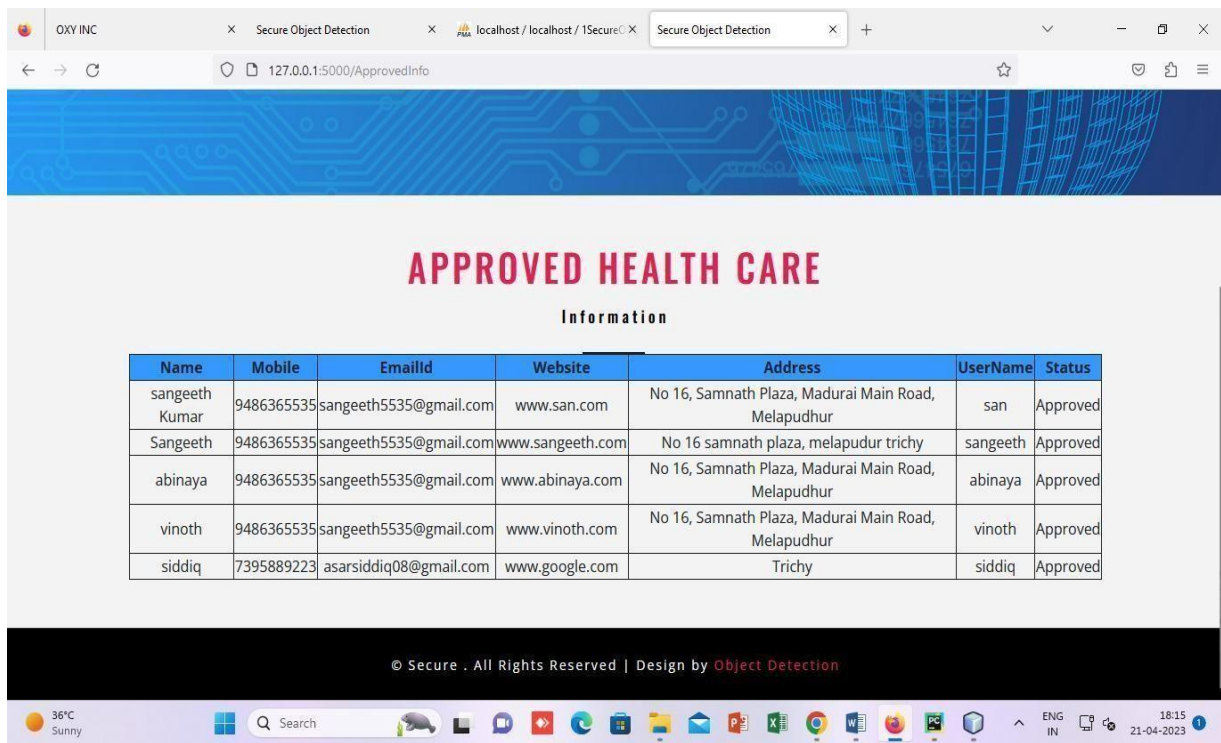


Fig 1.10 Approved Healthcare Details

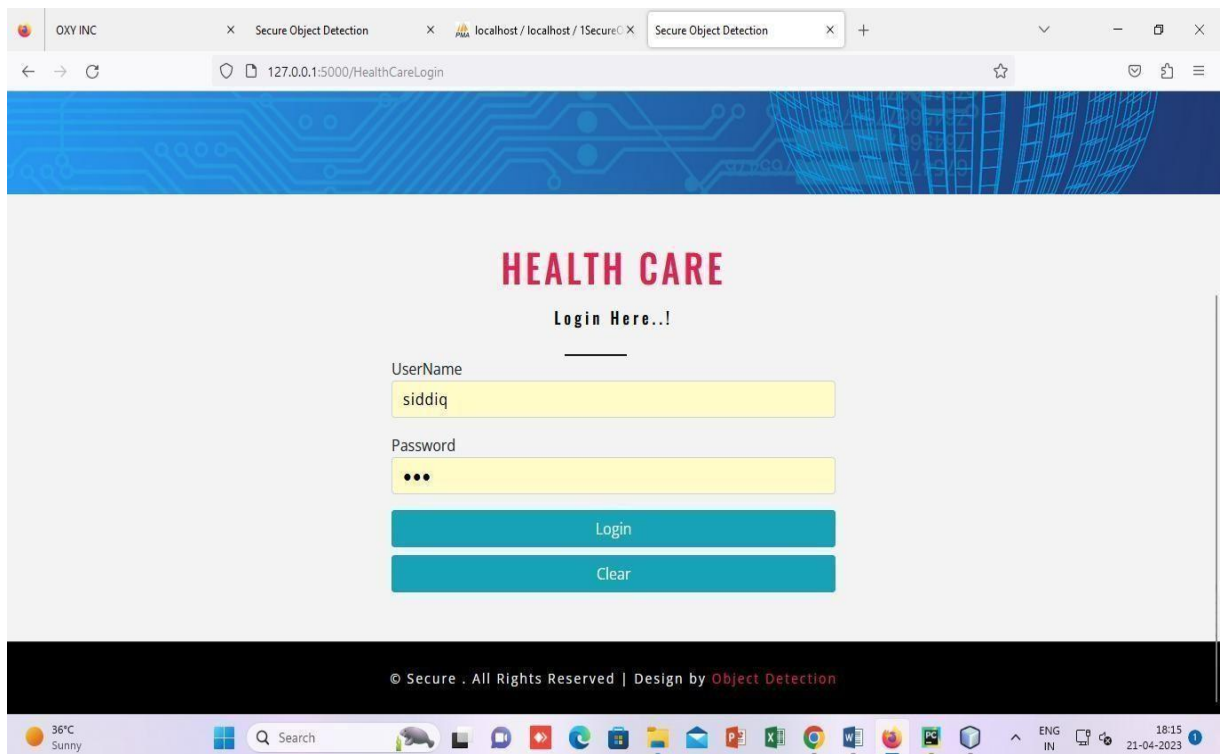


Fig 1.11 Healthcare Logi

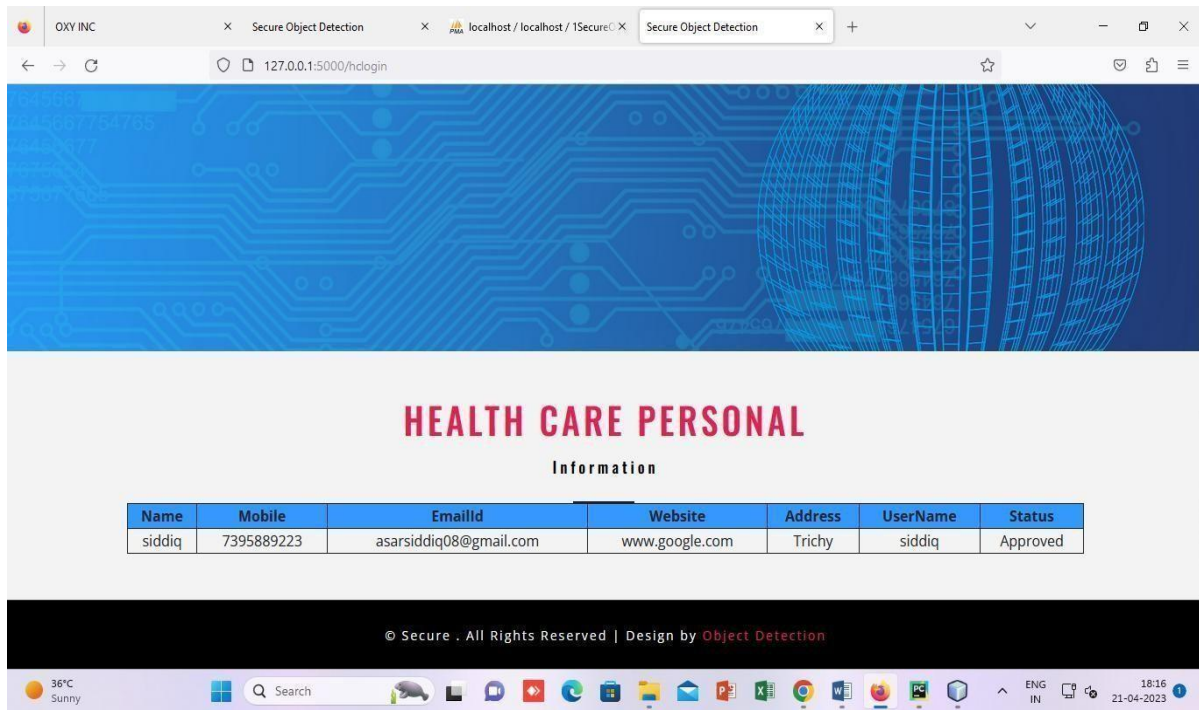


Fig 1.12 View Approved Status

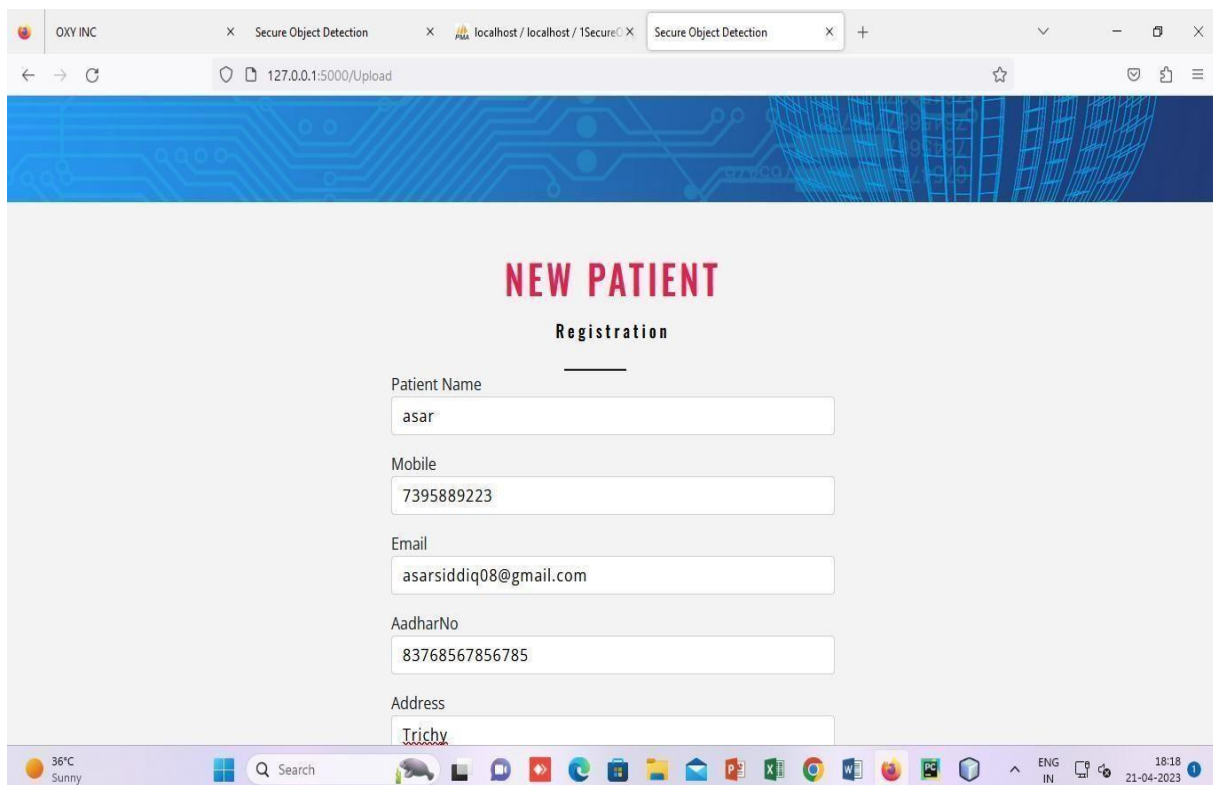


Fig 1.13 New Patient Register

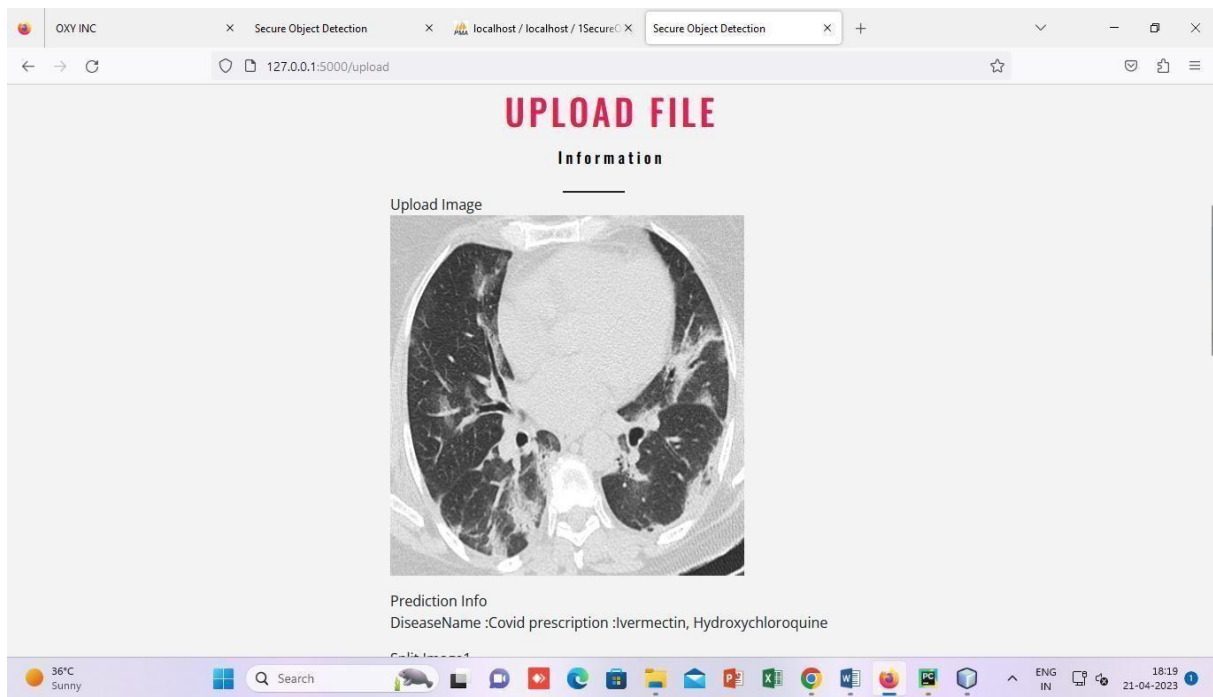


Fig 1.14 Upload Medical Image and Disease Prediction

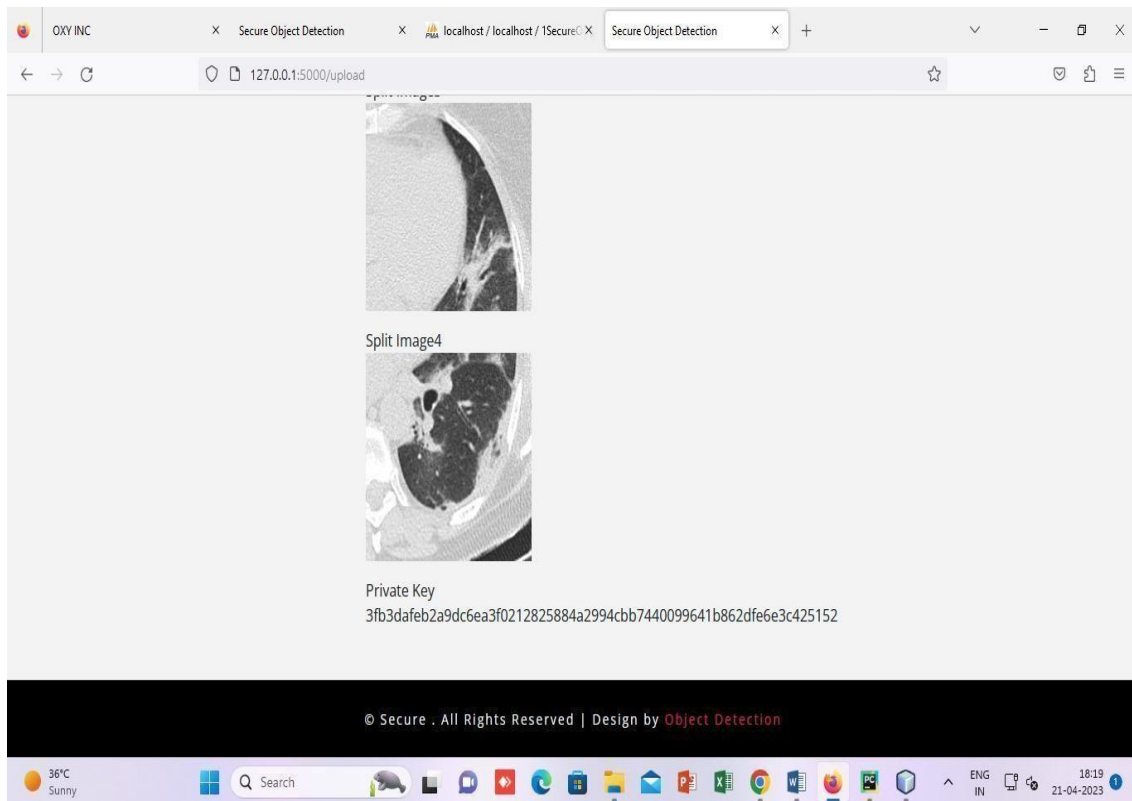


Fig 1.15 Image Split with Medical Information Hiding

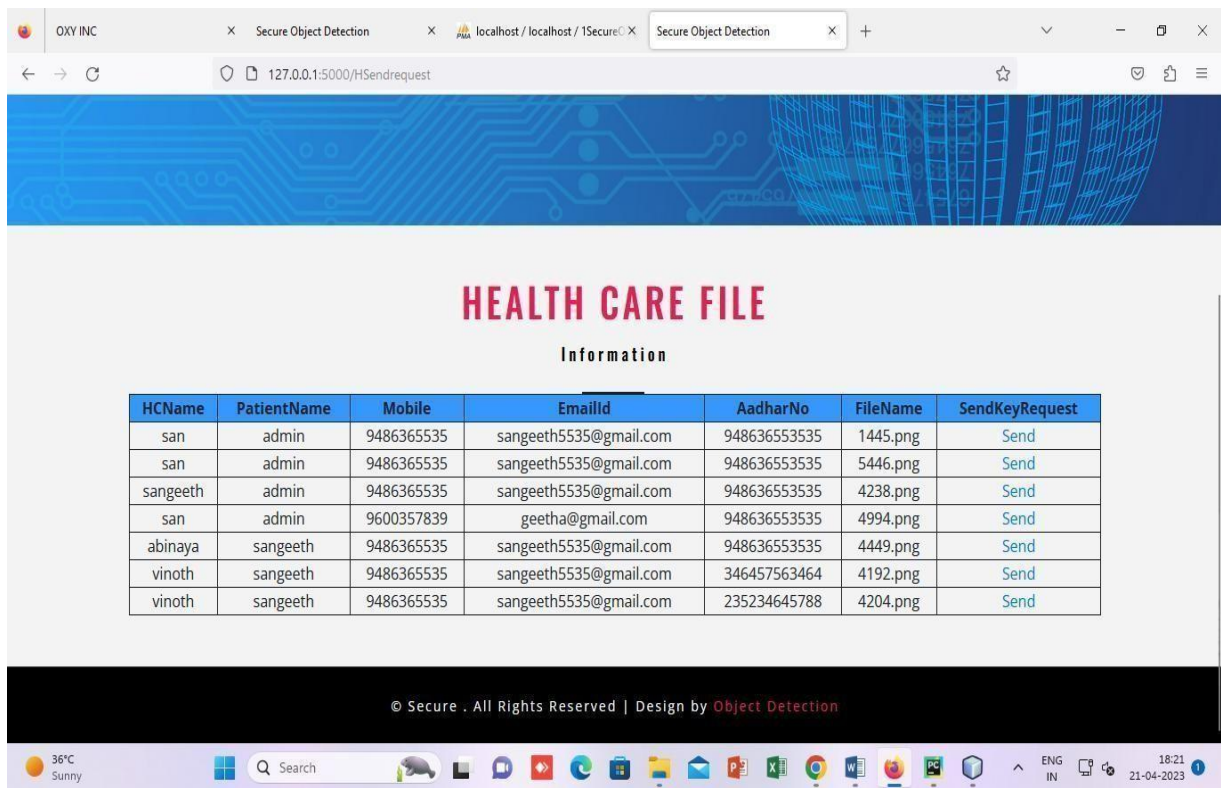


Fig 1.16 Send Medical Data

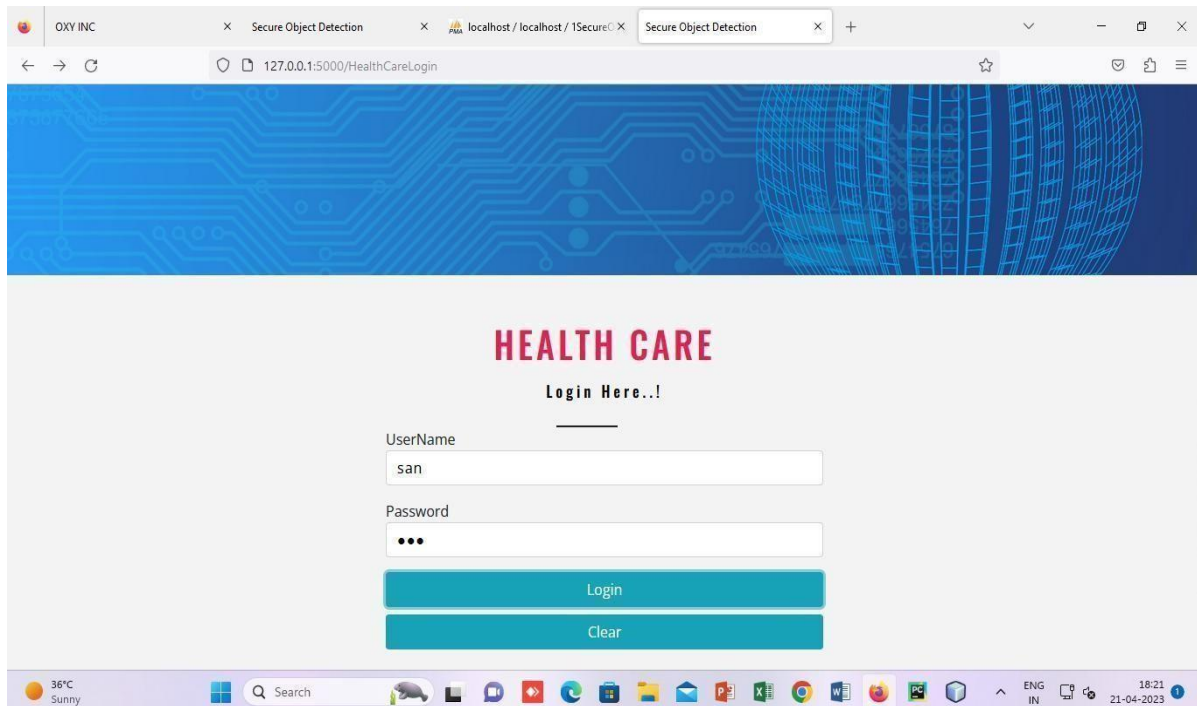


Fig 1.17 Patient Login

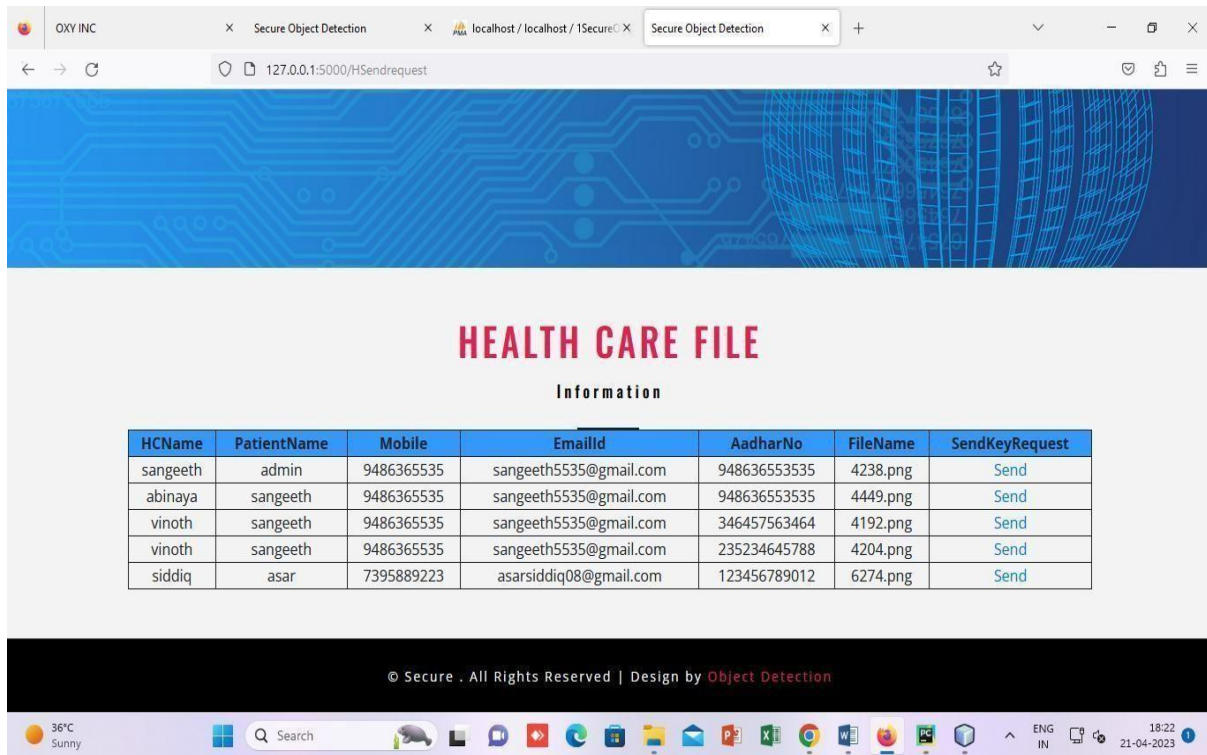


Fig 1.18 Send Key Request

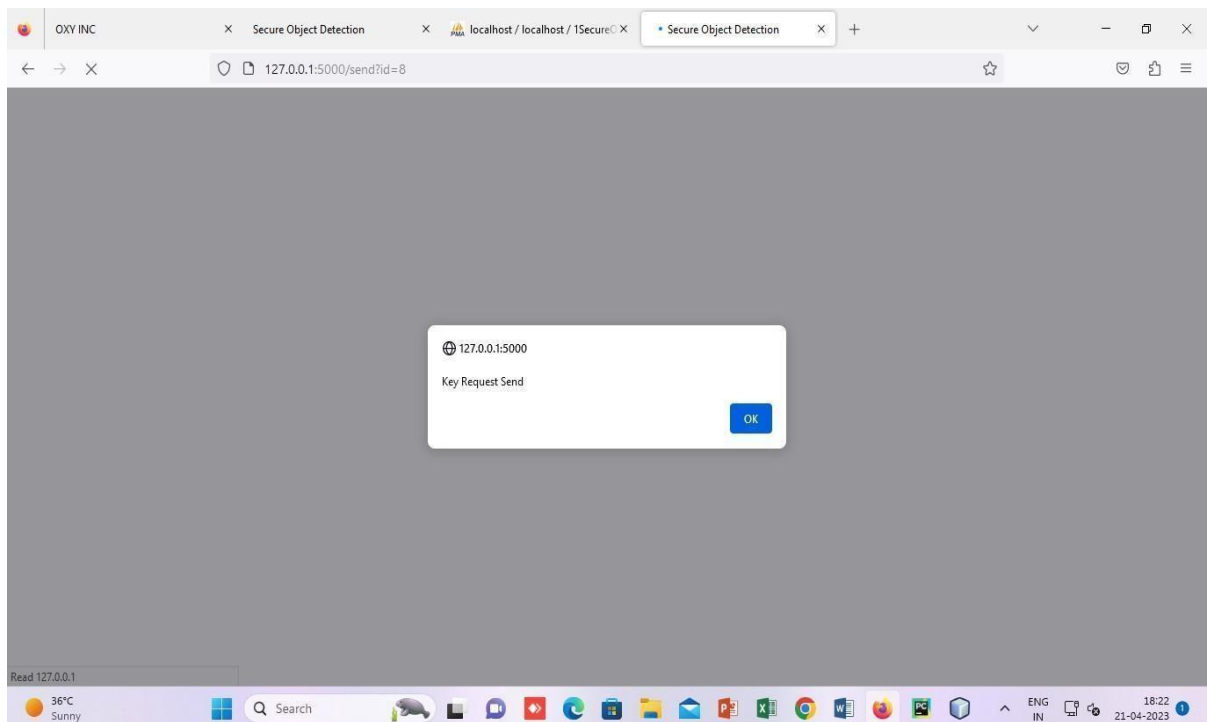


Fig 1.19 Key Request Sended

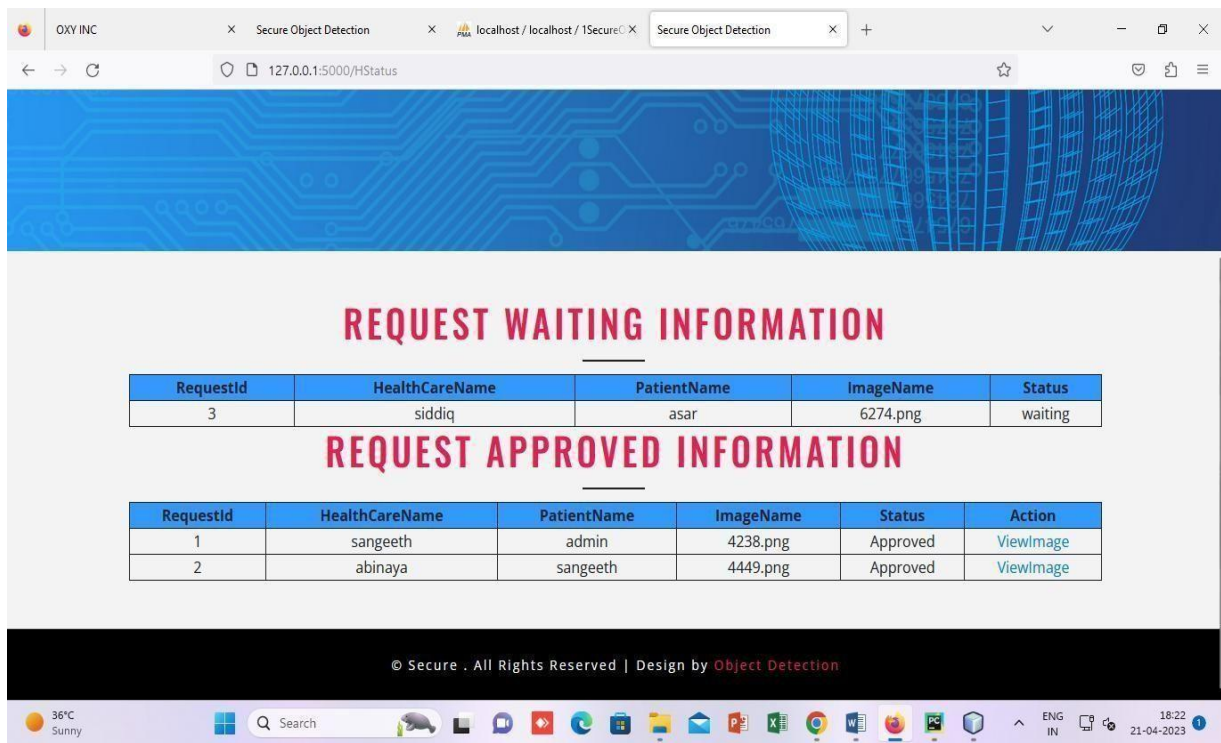


Fig 1.20 Waiting for Key Permission

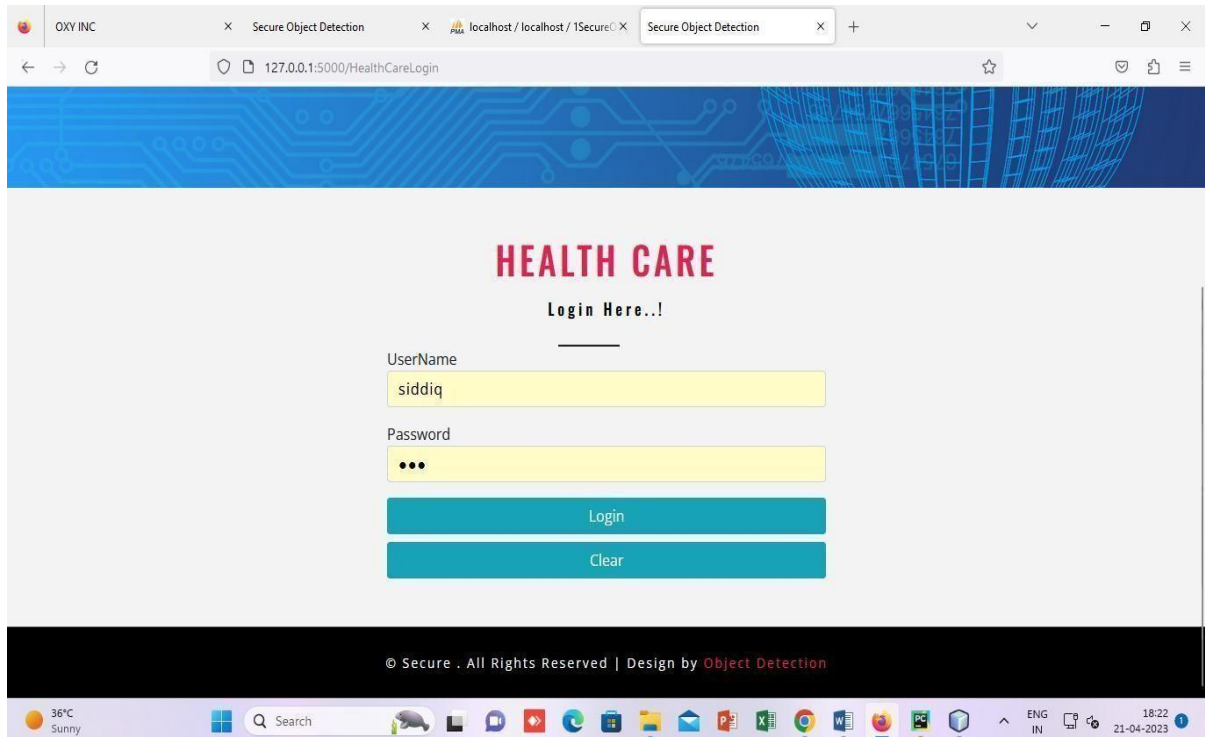


Fig 1.21 Healthcare Login

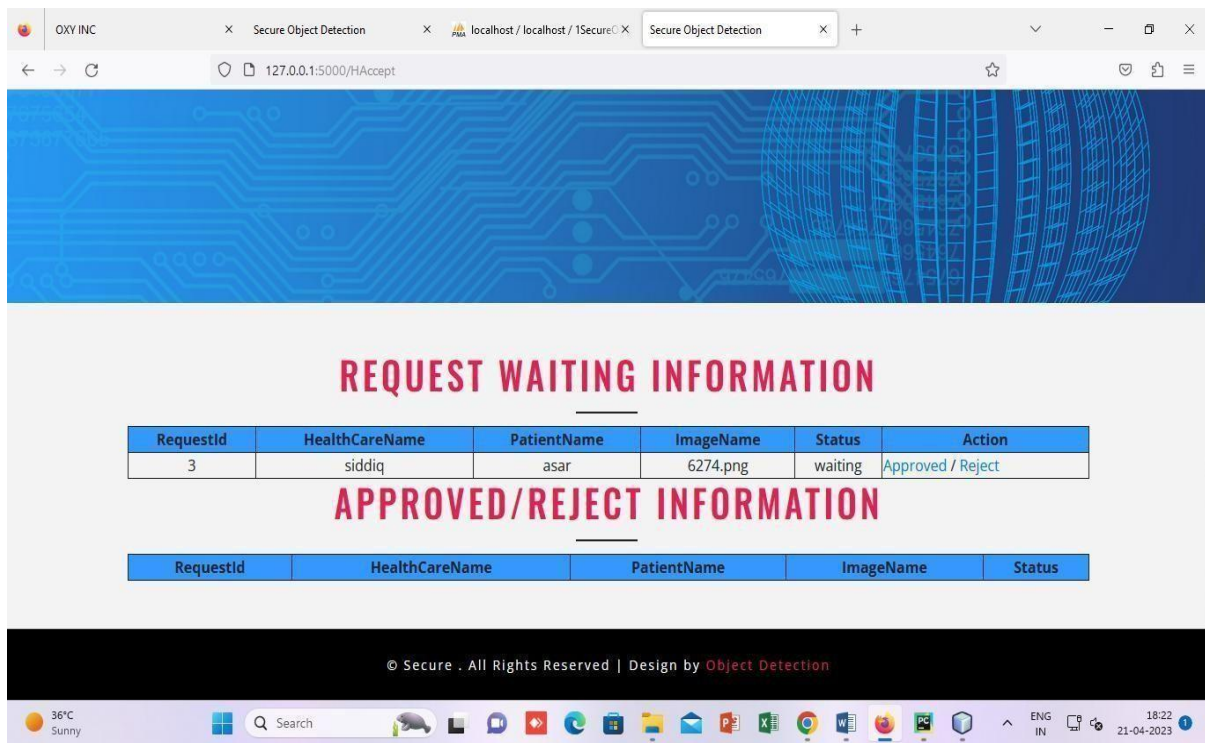


Fig 1.22 Approval for Key Request

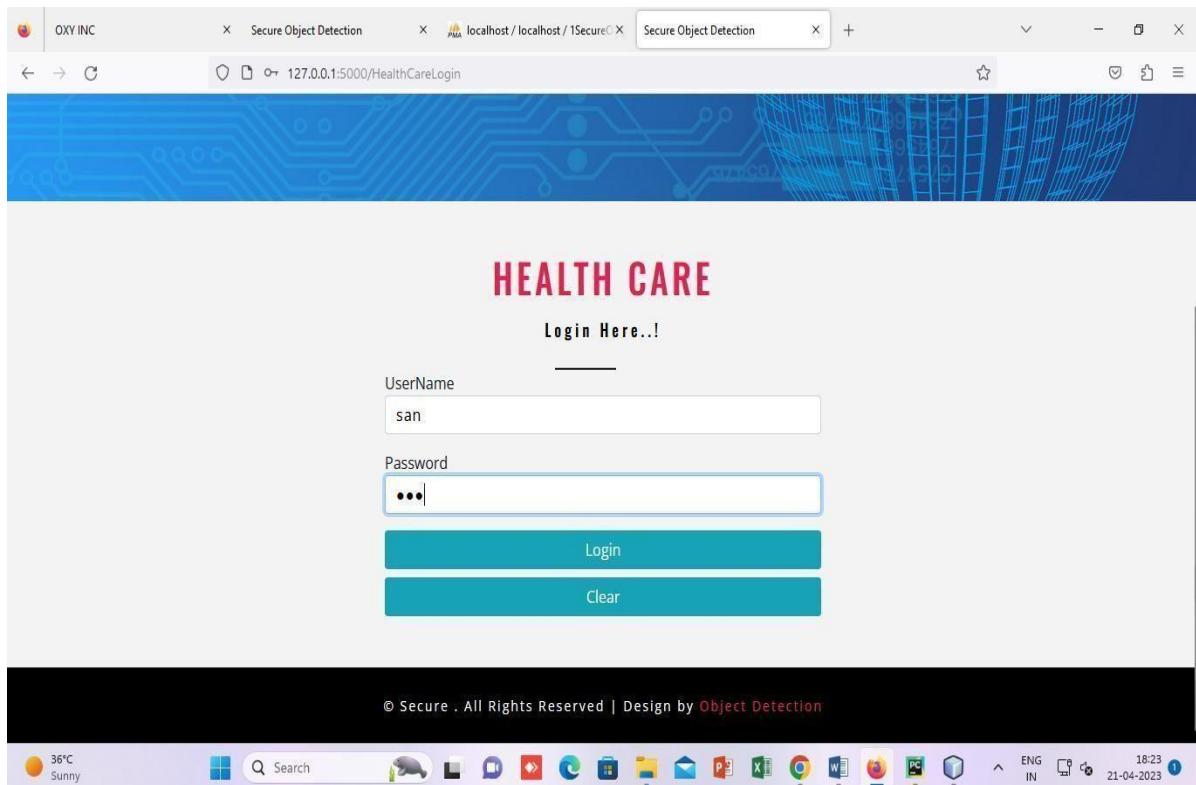


Fig 1.23 Patient Login

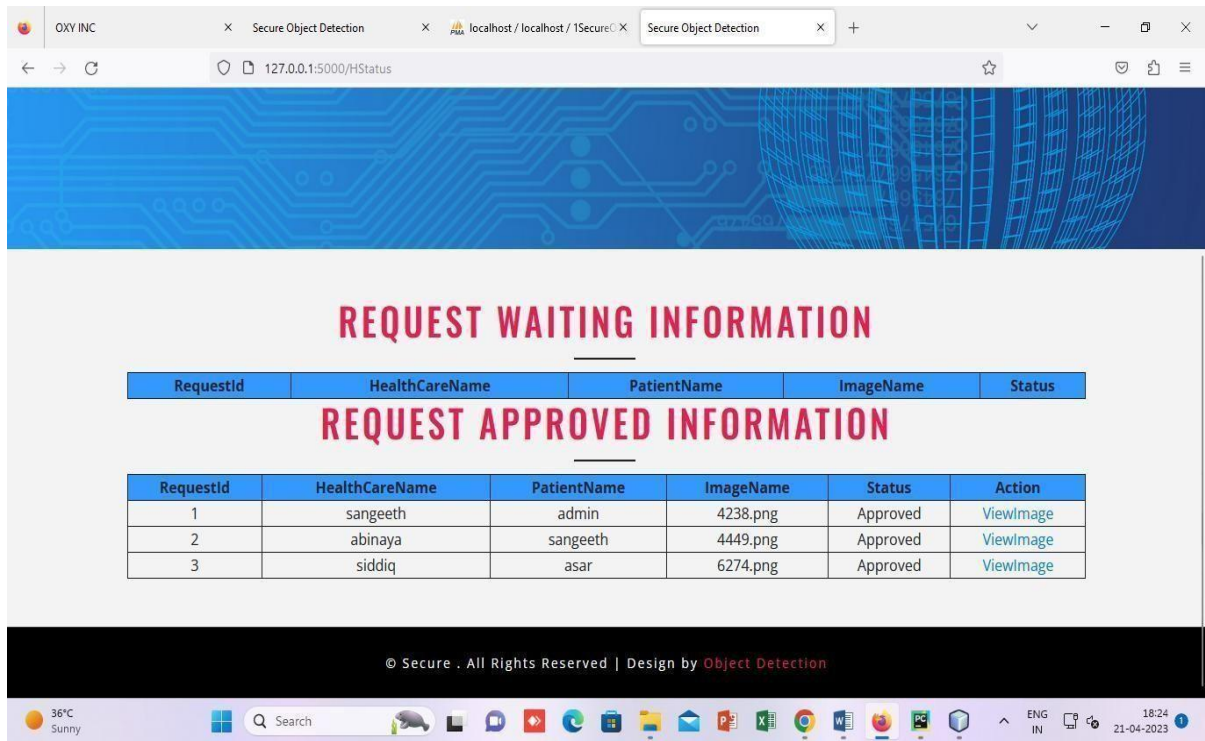


Fig 1.24 View Shared Medical Image

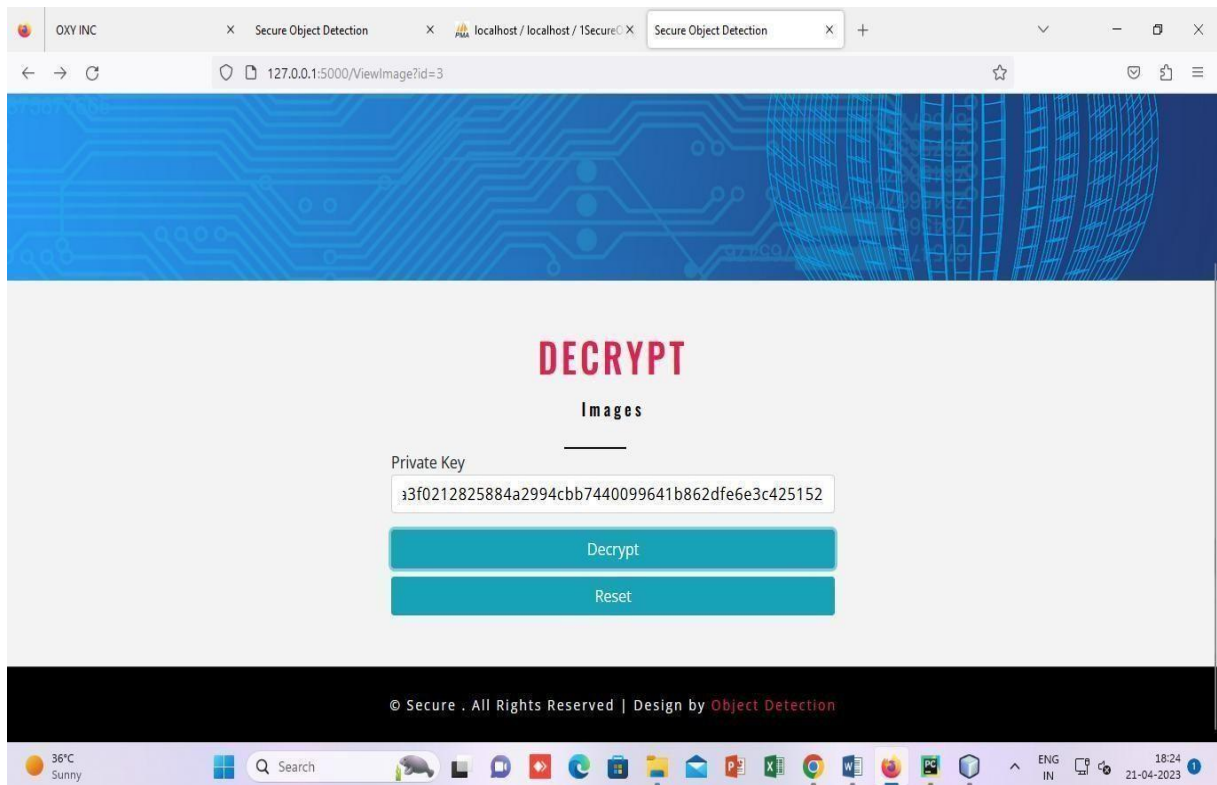


Fig 1.25 Secret Key Verification

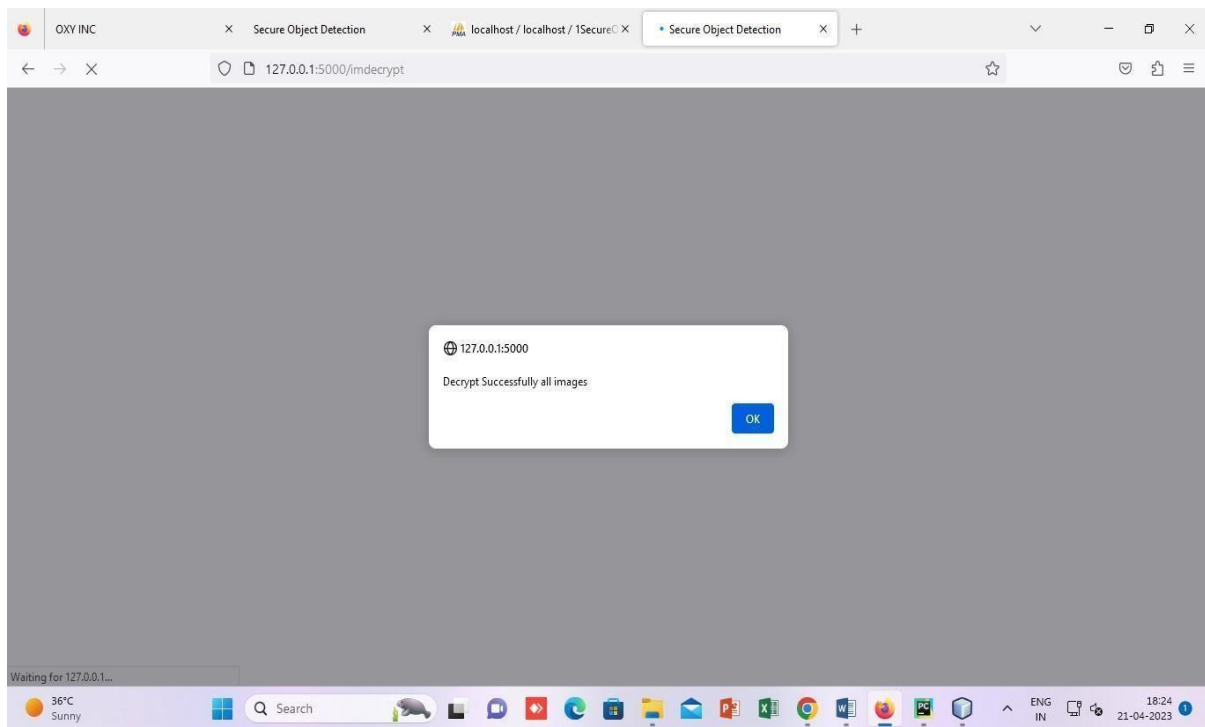


Fig 1.26 Image Shares Decryption

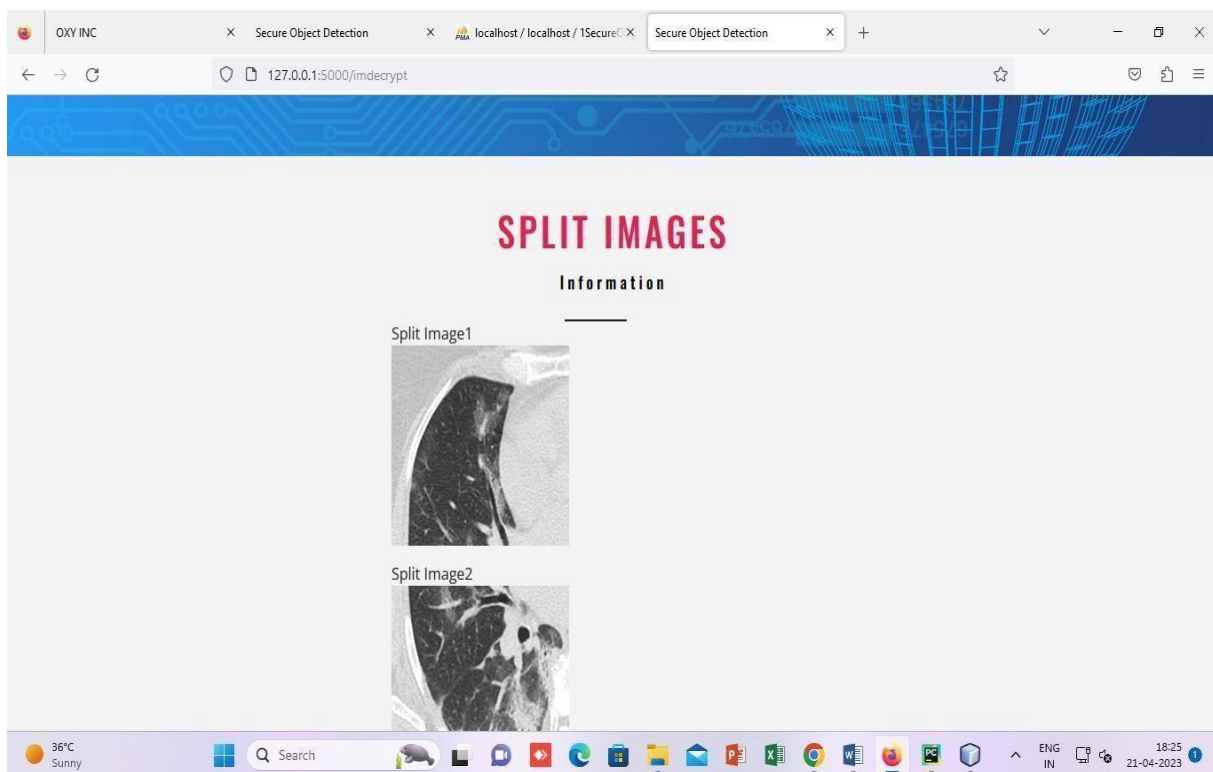


Fig 1.27 View Image Shares

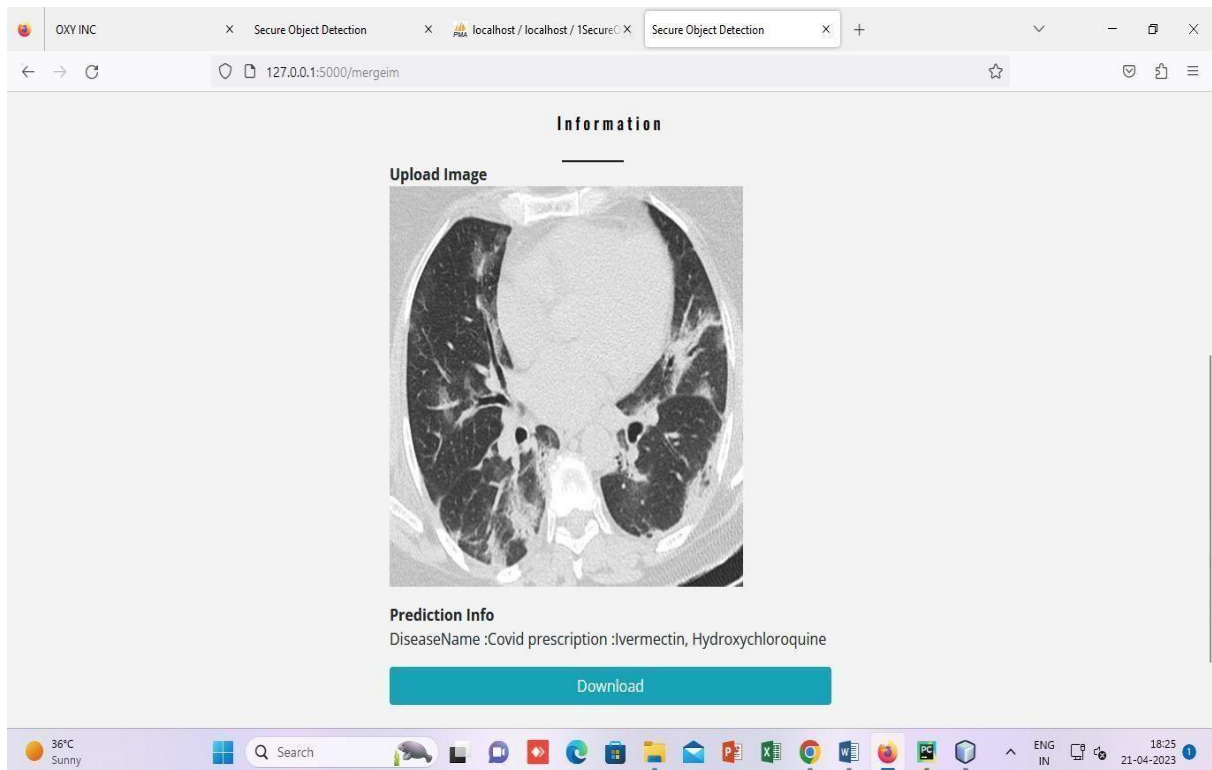


Fig 1.28 Share Reconstruction and Medical Data Extraction

REFERENCE

- [1] Aparna, Puvvadi, and Polurie Venkata Vijay Kishore. "Biometric based efficient medical image watermarking in E healthcare application." *IET Image Processing* 13, no. 3 (2019): 421-428.
- [2] Hasan, Mohammad Kamrul, Shayla Islam, Rossilawati Sulaiman, Sheroz Beiji Khan, Aisha-Hassan Abdalla Hashim, Shabana Habib, Mohammad Islam et al. "Lightweight encryption technique to enhance medical image security on internet of medical things applications. *IEEE Access* 9 (2021): 47731-47742.
- [3] Kamal, Sara T., Khalid M. Hosny, Taha M. Elgindy, Mohamed M. Darwish and Mostafa M. Fouda. "A new algorithm for grey and color medical images." *IEEE Access* 9 (2021): 37855-37865.
- [4] Liu, Xiyao, Jieting Lou, Hui Fang, Yan Chen, Pingbo Ouyang, Yifan Beiji Zou, and Lei Wang. "A novel robust reversible watermarking scheme for protecting authenticity and integrity of medical images." *Ieee Access* 7 (2019): 76580-76598.
- [5] Li, Zhuoling, Minghui Dong, Shiping Wen, Xiang Hu, Pan Zhou, and Zhigang Zeng. "CLU-CNNs: Object detection for medical images." *Neurocomputing* 350 (2019): 53-59.
- [6] Li, Xin, and Dongxiao Zhu. "Robust detection of adversarial attacks on medical images." In *2020 IEEE 17th International Symposium on Biomedical Imaging (ISBI)*, pp. 1154-1158. IEEE, 2020.
- [7] Masood, Fawad, Maha Driss, Wadii Boulila, Jawad Ahmad, Sadaqat Ur Rehman, Sana Ullah Jan, Abdullah Qayyum, and William J. Buchanan. "A lightweight chaos-based medical image encryption scheme using random shuffling and XOR operations." *Wireless Personal Communications* (2021): 1- 28.

- [8] Patel, Vishal. "A framework for secure and decentralized sharing of medical imaging data via blockchain consensus." *Health informatics journal* 25, no. 4 (2019): 1398-1411.
- [9] Qayyum, Adnan, Junaid Qadir, Muhammad Bilal, and Ala Al- Fuqaha "Secure and robust machine learning for healthcare: A survey." *IEEE Reviews in Biomedical Engineering* 14 (2020): 156-180.
- [10] Zhou, Yi, Xiaodong He, Lei Huang, Li Liu, Fan Zhu, Shanshan Cui, and Ling Shao. "Collaborative learning of semi-supervised segmentation and classification for medical images." In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 2079-2088. 2019.